1. What is parallel processing?

In computers, parallel processing is the ***processing of program instructions by dividing them among multiple processors with the objective of running a program in less time***.

In the earliest computers, only one program ran at a time. A computation-intensive program that took one hour to run and a tape copying program that took one hour to run would take a total of two hours to run.

First step of improvement: interleaving execution

An early form of parallel processing allowed the interleaved execution of both programs together. The computer would start an I/O operation, and while it was waiting for the operation to complete, it would execute the processor-intensive program. The total execution time for the two jobs would be a little over one hour.

Second step of improvement: Multiprogramming

The next improvement was multiprogramming. In a multiprogramming system, multiple programs submitted by users were each allowed to use the processor for a short time. To users it appeared that all of the programs were executing at the same time. Explicit requests for resources led to the problem of the deadlock.

Third step of improvement: Vector Processing

Vector processing was another attempt to increase performance by doing more than one thing at a time. In this case, capabilities were added to machines to allow a single instruction to add (or subtract, or multiply, or otherwise manipulate) two arrays of numbers. This was valuable in certain engineering applications where data naturally occurred in the form of vectors or matrices. In applications with less well-formed data, vector processing was not so valuable.

Fourth step of improvement: Multiprocessing

The next step in parallel processing was the introduction of multiprocessing. In these systems, two or more processors shared the work to be done. The earliest versions had a master/slave configuration. One processor (the master) was programmed to be responsible for all of the work in the system; the other (the slave) performed only those tasks it was assigned by

the master. This arrangement was necessary because it was not then understood how to program the machines so they could cooperate in managing the resources of the system

What is the difference between a multi-processor system and a multicore processor system?

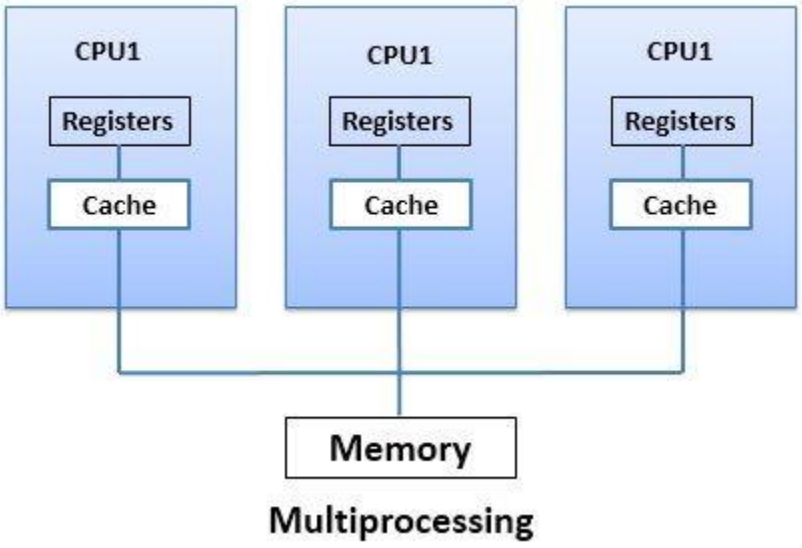
A *multiprocessor* system contains *more than one* CPU (also known as processor) and they work in parallel. This is called Simultaneous Multiprocessing (SMP). Multiprocessor systems have a special type of motherboard which has several CPU sockets.

A *multicore* system contains more than one execution core on *one* CPU. This is called Chip-level Multiprocessing (CMP). A multicore chip can have a dedicated execution unit and L1 cache for a core and a shared L2 cache for the entire CPU.

Difference Between Multiprocessing and Multithreading

Definition of Multiprocessing

A multiprocessing system is one which has more than two processors. The CPUs are added to the system to increase the computing speed of the system. Each CPU has its own set of registers and main memory. Just because CPUs are separate, it may happen that one CPU must not have anything to process and may sit idle and the other may be overloaded with the processes. In such cases, the processes and the resources are shared dynamically among the processors.



Multiprocessing

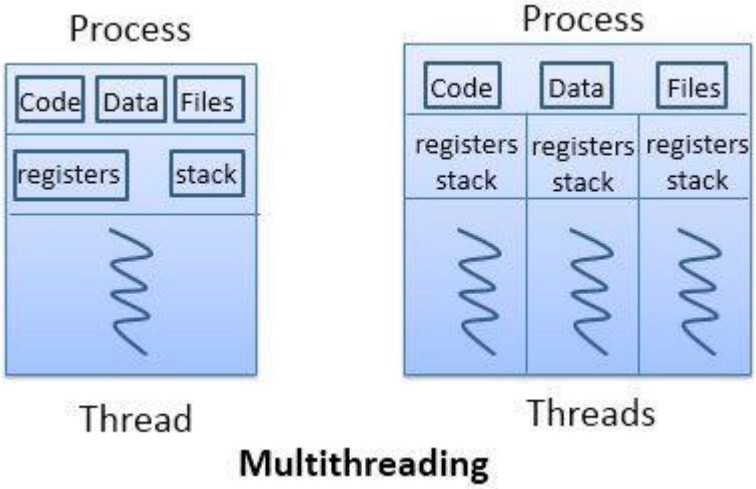
Multiprocessing can be classified as **symmetric multiprocessing** and **asymmetric multiprocessing**. Multiprocessing can also be **uniform memory access** type or **non-uniform memory access** type.

Definition of Multithreading

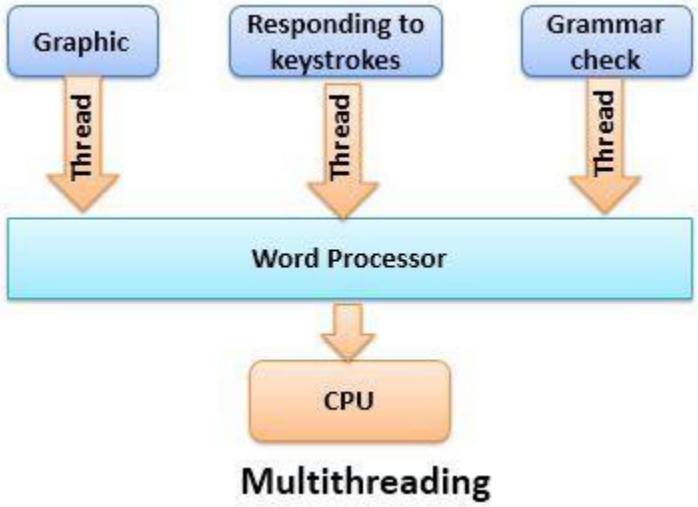
Multithreading is the execution of multiple threads of a single process concurrently within the context of that process.

Now let us first discuss what is a thread?

A **thread** (also called lightweight process) of a process means a code segment of a process, which has its own thread ID, program counter, registers and stack and can execute independently. But threads belonging to the same process has to share the belongings of that process like code, data, and system resources.



To understand the multithreading concept let us take an **example** of a word processor. A word processor, displays graphic, responds to keystrokes, and at the same time, it continues spelling and grammar checking. You do not have to open different word processors to do this concurrently. It does get happen in a single word processor with the help of multiple threads.



Now let us take into consideration the benefits of multithreading. Multithreading increases the **responsiveness** as if one thread of a process is blocked or performing the lengthy operation, the process still continues. The second benefit of multithreading is **resource sharing** as several threads of a process share same code and data within the same address space.

Creating a thread is **economical** as it shares the code and data of the process to which they belong. So the system does not have to allocate resources separately for each thread. Multithreading can be **increased** on multiprocessing operating system. As multithreading on multiple CPUs increases **parallelism**.

Comparison Chart

BASIS FOR COMPARISON	**MULTIPROCESSING**	**MULTITHREADING**
Basic	Multiprocessing adds CPUs to increase computing power.	Multithreading creates multiple threads of a single process to increase computing power.
Execution	Multiple processes are executed concurrently.	Multiple threads of a single process are executed concurrently.
Creation	Creation of a process is time-consuming and resource intensive.	Creation of a thread is economical in both sense time and resource.

BASIS FOR COMPARISON	MULTIPROCESSING	MULTITHREADING
Classification	Multiprocessing can be symmetric or asymmetric.	Multithreading is not classified.

Types of Thread

Threads are implemented in following two ways −

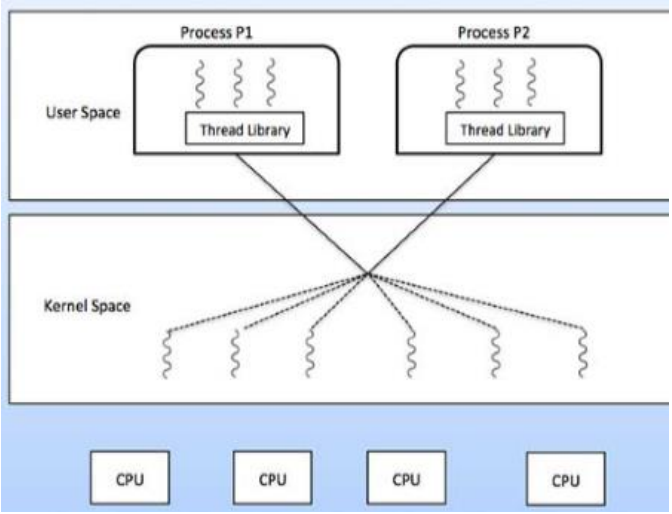
- User Level Threads − User managed threads.
- Kernel Level Threads − Operating System managed threads acting on kernel

Multithreading models are three types

- Many to many relationships.
- Many to one relationship.
- One to one relationship.

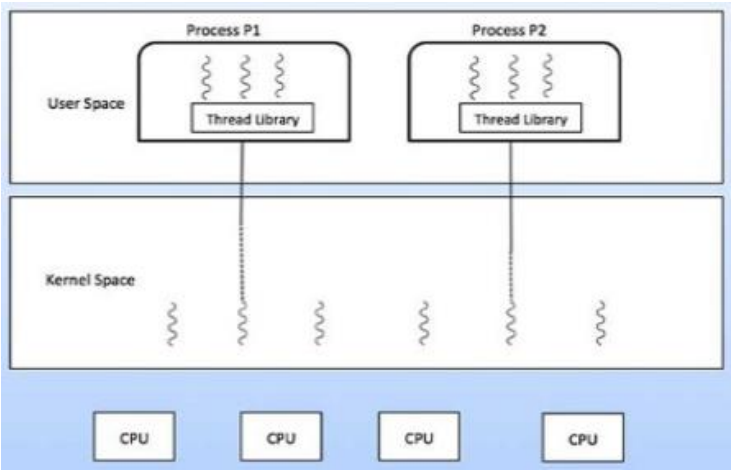
Many to Many Model

- The many-to-many model multiplexes any number of user threads onto an equal or smaller number of kernel threads.
- The following diagram shows the many-to-many threading model where 6 user level threads are multiplexing with 6 kernel level threads. In this model, developers can create as many user threads as necessary and the corresponding Kernel threads can run in parallel on a multiprocessor machine.

- 

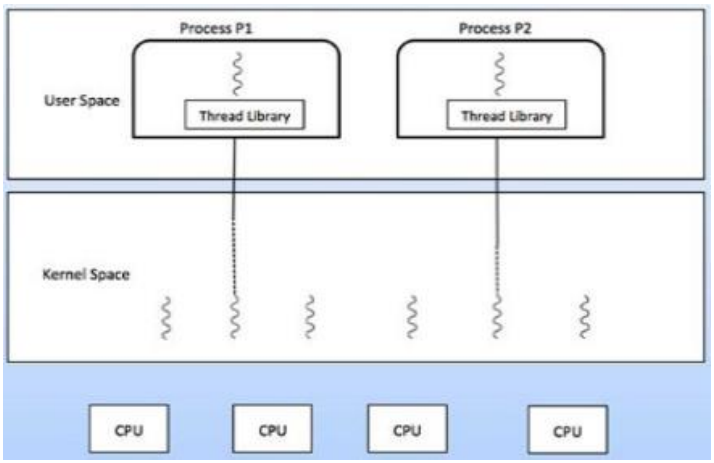
Many to One Model

- Many-to-one model maps many user level threads to one Kernel-level thread. Thread management is done in user space by the thread library. When thread makes a blocking system call, the entire process will be blocked. Only one thread can access the Kernel at a time, so multiple threads are unable to run in parallel on multiprocessors.
- If the user-level thread libraries are implemented in the operating system in such a way that the system does not support them, then the Kernel threads use the many-to-one relationship modes.



One to One Model

- There is one-to-one relationship of user-level thread to the kernel-level thread.

- 

Explain Flynn's **classification.**

Flynn classified architecture in terms of streams of data and instructions. The essence of the idea is that computing activity in every machine is based on:

i. Stream of instructions, i.e., sequence of instructions executed by the machine.

ii. Stream of data, i.e., sequence of data including input, temporary or partial results referred by the instructions.

Further a machine may have a single or multiple streams of each kind. Based on these, computer architecture is characterized by the multiplicity of the hardware to serve instructions and data streams as follows:

I. SISD: Single Instruction and single data stream

II. SIMD: Single Instruction and Multiple Data Stream

III. MIMD: Multiple Instruction and Multiple Data Stream

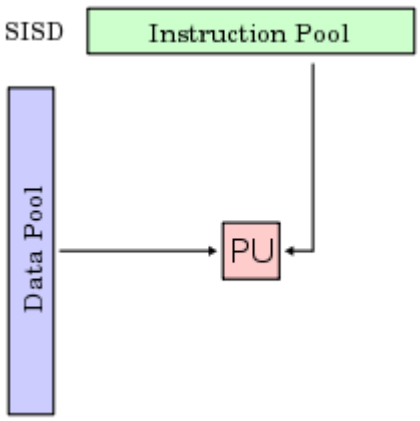
IV. MISD: Multiple Instruction and Single Data Stream

SISD (Single Instruction, Single Data stream)

Single Instruction, Single Data (SISD) refers to an Instruction Set Architecture in which a single processor (one CPU) executes exactly one instruction stream at a time and also fetches or stores one item of data at a time to operate on data stored in a single memory unit.

Most of the CPU design, based on the von Neumann architecture, from the beginning till recent times are based on the SISD.

The SISD model is a typical non-pipelined architecture with the general-purpose registers, as well as dedicated special registers such as the Program Counter (PC), the Instruction Register (IR), Memory Address Registers (MAR) and Memory Data Registers (MDR).

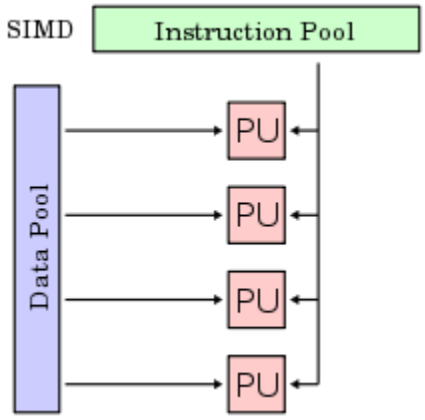


SIMD (Single Instruction, Multiple Data streams)

Single Instruction, Multiple Data (SIMD) is an Instruction Set Architecture that have a single control unit (CU) and more than one processing unit (PU) that operates like a von Neumann machine by executing a single instruction stream over PUs, handled through the CU.

The CU generates the control signals for all of the PUs and by which executes the same operation on different data streams. The SIMD architecture, in effect, is capable of achieving data level parallelism just like with vector processor.

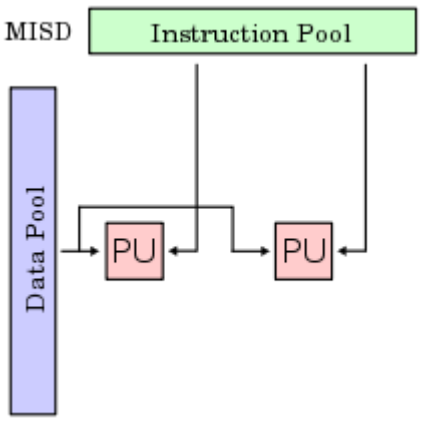
Some of the examples of the SIMD based systems include IBM's AltiVec and SPE for PowerPC, HP's PA-RISC Multimedia Acceleration eXtensions (MAX), Intel's MMX and iwMMXt, SSE, SSE2, SSE3 and SSSE3, AMD's 3DNow! etc.



MISD (Multiple Instruction, Single Data stream)

Multiple Instruction, Single Data (MISD) is an Instruction Set Architecture for parallel computing where many functional units perform different operations by executing different instructions on the same data set.

This type of architecture is common mainly in the fault-tolerant computers executing the same instructions redundantly in order to detect and mask errors.

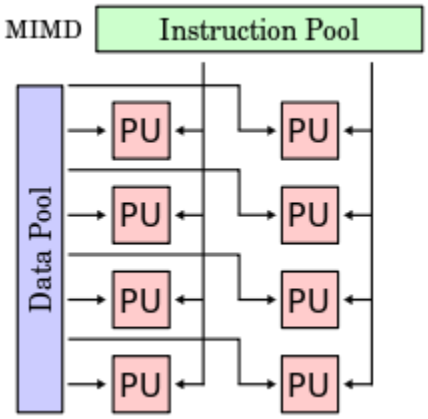


MIMD(Multiple Instruction, Multiple Data streams)

Multiple Instruction stream, Multiple Data stream (MIMD) is an Instruction Set Architecture for parallel computing that is typical of the computers with multiprocessors. Using the MIMD, each processor in a multiprocessor system can execute asynchronously different set of the instructions independently on the different set of data units.

The MIMD based computer systems can use the shared memory in a memory pool or work using distributed memory across heterogeneous network computers in a distributed environment.

The MIMD architectures is primarily used in a number of application areas such as computer-aided design/computer-aided manufacturing, simulation, modeling, communication switches etc.



What is the difference between SIMD and MIMD in parallel computing?

SIMD and MIMD are two types of parallel computing architectures.

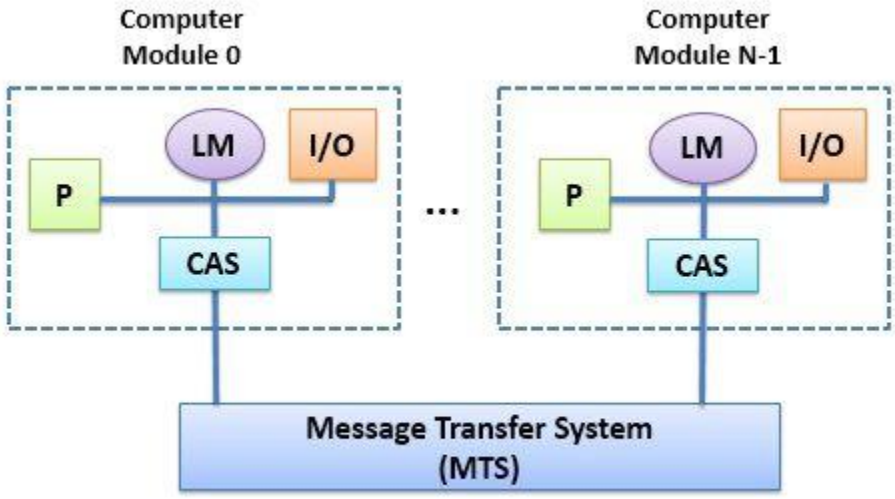
In SIMD processor, we have a global controller unit with many processors and all the processor does the same thing. Like in the case of vector addition, all the processor is assigned a data to

add and all of them will be doing the addition operations. Thus, SIMD is used in the situations when a lot of computations are required with all the processors doing the same job. Furthermore, since SIMD is simple to design and is less expensive and faster. Being synchronous, programming in SIMD architecture is comparatively easier.

SIMD is limited to only few applications. In the cases like where we don't have identical but many independent tasks then we use MIMD architecture. Here, each processor will be doing an independent task. Like one processor many be doing the addition, another multiplication and so on. MIMD operations may or may not be synchronized. However, because of the complexity MIMD processors tend to be expensive.

Loosely Coupled vs tightly coupled Multiprocessor System

Multiprocessor is one which has more than two processors in the system. Now when the **degree of coupling** between these processors is very **low**, the system is called **loosely coupled multiprocessor system**. In loosely coupled system each processor has its **own local memory, a set of input-output device**s and a **channel and arbiter switch (CAS)**.



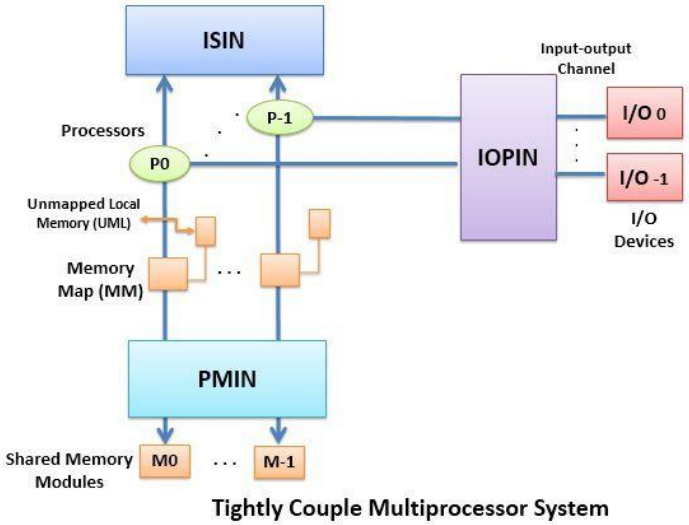
Loosely Couple Multiprocessor System

Processes that execute on different computer modules communicate with each other by exchanging the **messages** through a physical segment of **message transfer system (MTS)**. The loosely coupled system is also known as **distributed system**. The loosely coupled system is **efficient** when the processes running on different computer module require **minimal interaction**.

If the request for accessing MTS of two or more computer module collide, the **CAS responsibly** chooses one of the simultaneous requests and delay other requests until selected request is serviced completely. The CAS has a **high-speed communication memory** which can be accessed by all the processors in the system. The communication memory in CAS is used to **buffer the transfers of messages**.

Definition of Tightly Coupled Multiprocessor System

The **throughput** of the loosely coupled system may be **too low** for some of the applications that require **fast access time**. In this case, **Tightly coupled microprocessor system** must be used. The tightly coupled system has **processors**, **shared memory modules**, **input-output channels**.



Tightly Couple Multiprocessor System

The above units of the tightly coupled system are connected through the set of three **interconnection networks**, **processor-memory interconnection network (PMIN)**, **I/O-processor interconnection network (IOPIN)** and the **interrupt-signal interconnection network (ISIN)**. The use of these three interconnection networks is as follow.

PMIN: It is a switch which **connects** each **processor** to every **memory module**. It can also be designed in a way that a processor can broadcast data to one or more memory module.

ISIN: It allows each **processor** to **direct an interrupt** to any **other processor**.

IOPIN: It allows a **processor** to **communicate** with an **I/O channel** which is connected to input-output devices.

The loosely coupled system has a distributed memory which delays the data rate whereas, the tightly coupled system has shared memory which increases the data rate.

Comparison Chart

BASIS FOR COMPARISON	**LOOSELY COUPLED MULTIPROCESSOR SYSTEM**	**TIGHTLY COUPLED MULTIPROCESSOR SYSTEM**
Basic	Each processor has its own memory module.	Processors have shared memory modules.
Efficient	Efficient when tasks running on different processors, has minimal interaction.	Efficient for high-speed or real-time processing.
Memory conflict	It generally, do not encounter memory conflict.	It experiences more memory conflicts.

BASIS FOR COMPARISON	LOOSELY COUPLED MULTIPROCESSOR SYSTEM	TIGHTLY COUPLED MULTIPROCESSOR SYSTEM
Interconnections	Message transfer system (MTS).	Interconnection networks PMIN, IOPIN, ISIN.
Data rate	Low.	High.
Expensive	Less expensive.	More expensive.

SMP (symmetric multiprocessing)

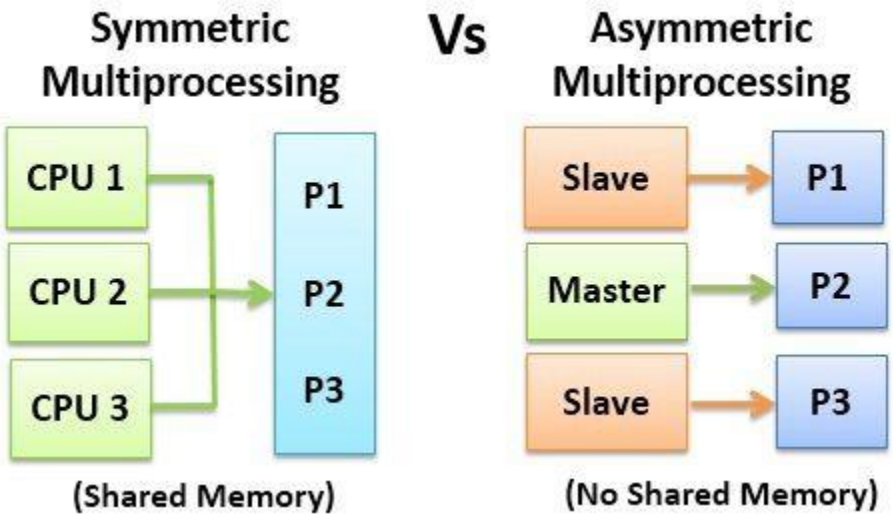
SMP (symmetric multiprocessing) is the processing of programs by multiple processors that share a common operating system and memory. In symmetric (or "tightly coupled") multiprocessing, the processors share memory and the I/O bus or data path. A single copy of the operating system is in charge of all the processors. SMP, also known as a "shared everything" system, does not usually exceed 16 processors.
A symmetric multiprocessor (SMP) can be defined as a standalone computer system with the following characteristic:

1. There are two or more similar processor of comparable capability.
2. These processors share the same main memory and IO facilities and are interconnected by a bus or other internal connection scheme.
3. All processors share access to IO devices, either through the same channels or through different channels that provide paths to the same device.
4. All processors can perform the same functions.
5. The system is controlled by an integrated operating system that provides interaction between processors and their programs at the job, task, file and data element levels.

The operating system of a SMP schedules processors or thread across all of the processors. SMP has a potential advantage over uniprocessor architecture:

- Performance: A system with multiple processors will perform in a better way than one with a single processor of the same type if the task can be organized in such a manner that some portion of the work done can be done in parallel.
- Availability: Since all the processors can perform the same function in a systematic multiprocessor, the failure of a single processor does not stop the machine. Instead, the system can continue to function at reduce performance level.
- Incremental growth: A user can enhance the performance of a system by adding an additional processor.
- Sealing: Vendors can offer a range of product with different price and performance characteristics based on number of processors configured in the system.

Difference Between Symmetric and Asymmetric Multiprocessing



Definition of Symmetric Multiprocessing

Symmetric Multiprocessing is one in which all the processor run the tasks in the operating system. It has **no master-slave** relationship like asymmetric multiprocessing. All the processors here, communicate using the **shared memory**.

The processors start executing the processes from the common ready queue. Each processor may also have its own private queue of ready processes to get executed. It must be taken care by the **scheduler** that no two processors execute the same process.

Symmetric Multiprocessing has proper **load balancing**, better **fault tolerance** and also reduces the chance of CPU **bottleneck**. It is **complex** as the memory is shared among all the processors. In Symmetric Multiprocessing, a processor failure results in **reduced computing capacity**.

Definition of Asymmetric Multiprocessing

Asymmetric Multiprocessing has the **master-slave** relationship among the processors. There is one master processor that controls remaining slave processor. The master processor allots processes to slave processor, or they may have some predefined task to perform.

The master processor controls the **data structure**. The **scheduling** of processes, **I/O** processing and other system activities are controlled by the **master processor**.

In case a master processor fails, one processor among the slave processor is made the master processor to continue the execution. In case if a slave processor fails, the other slave processor take over its job. Asymmetric Multiprocessing is **simple** as there only one processor that is controlling data structure and all the activities in the system.

Comparison Chart

BASIS FOR COMPARISON	**SYMMETRIC MULTIPROCESSING**	**ASYMMETRIC MULTIPROCESSING**
Basic	Each processor run the tasks in Operating System.	Only Master processor run the tasks of Operating System.
Process	Processor takes processes from a common ready queue, or there may be a private ready queue for each processor.	Master processor assign processes to the slave processors, or they have some predefined processes.
Architecture	All processor in Symmetric Multiprocessing has the same architecture.	All processor in Asymmetric Multiprocessing may have same or different architecture.

BASIS FOR COMPARISON	SYMMETRIC MULTIPROCESSING	ASYMMETRIC MULTIPROCESSING
Communication	All processors communicate with another processor by a shared memory.	Processors need not communicate as they are controlled by the master processor.
Failure	If a processor fails, the computing capacity of the system reduces.	If a master processor fails, a slave is turned to the master processor to continue the execution. If a slave processor fails, its task is switched to other processors.

Multiprocessors increase the speed of the system, as one can execute multiple processes simultaneously. Asymmetric multiprocessing is simple, only one processor (master) can access the data structure. While Symmetric Multiprocessing is complex as the data structure is shared among all the processors and all the processor need to work in synchronization.

Discuss how multiprocessors can be organized?

The organization of multiprocessor system can be classified as follows:

- Time shared or common bus
- Multiport memory
- Central control unit.

Time shared Bus:

Time shared bus is the simplest mechanism for constructing a multiprocessor system. The bus consists of control, address and data lines. The block diagram is shown in Figure 10.2.

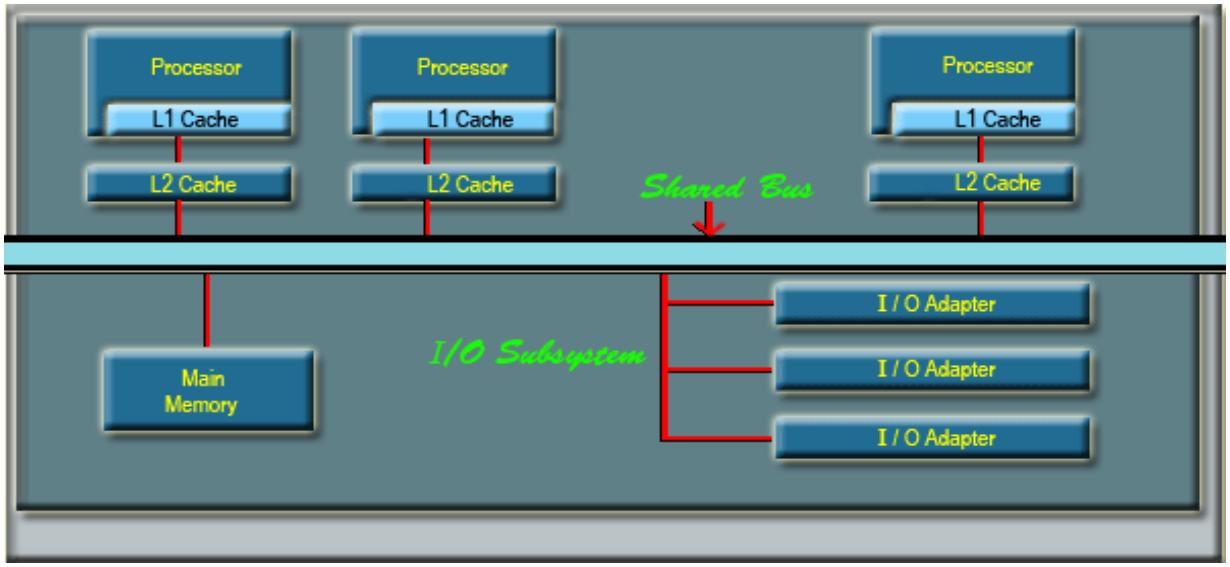


Figure 10.2: Time shared bus

The following features are provided in time-shared bus organization:

- Addressing: It must be possible to distinguish modules on the bus to determine the source and destination of data
- Arbitration: Any I/O module can temporarily function as "master". A mechanism is provided to arbitrate competing request for bus control, using some sort of priority scheme.
- Time shearing: when one module is controlling the bus, other modules are locked out and if necessary suspend operation until bus access in achieved.

The bus organization has several advantages compared with other approaches:

- Simplicity: This is the simplest approach to multiprocessor organization. The physical interface and the addressing, arbitration and time sharing logic of each processor remain the same as in a single processor system.
- Flexibility: It is generally easy to expand the system by attaching more processor to the bus.
- Reliability: The bus is essentially a passive medium and the failure of any attached device should not cause failure of the whole system.
- The main drawback to the bus organization is performance. Thus, the speed of the system is limited by the bus cycle time (*is the cycle or time required to make a single **read** or write transaction between the cpu and the external memory*.).

To improve performance, each processor can be equipped with local cache memory. The use of cache leads to a new problem which is known as cache coherence problem.

Multiport Memory:

The multiport memory approach allows the direct, independent access of main memory modules by each processor and io module.
The multiport memory system is shown in Figure 10.3

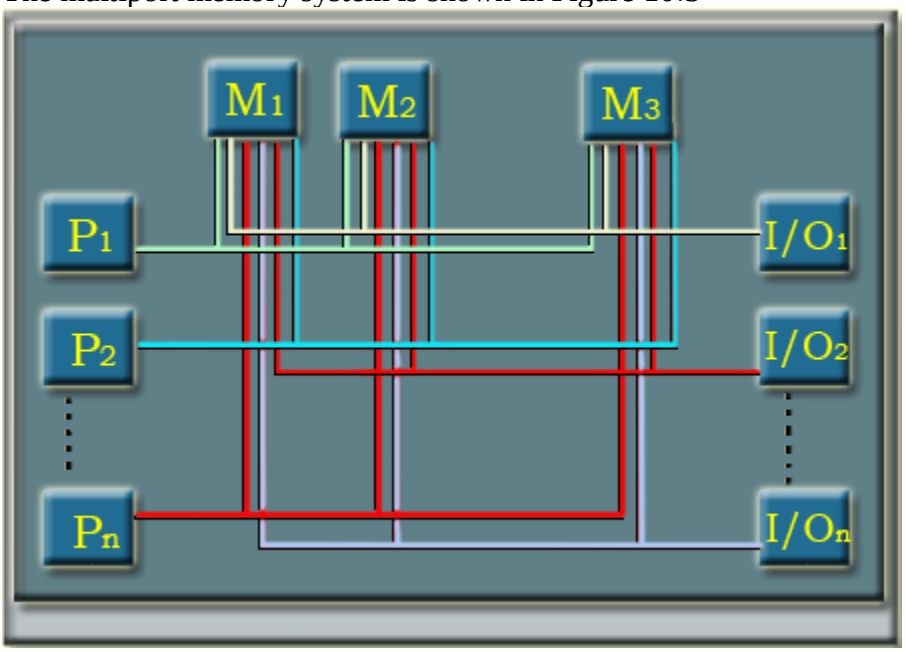


Figure 10.3: Multiport memory

The multiport memory approach is more complex than the bus approach, requiring a fair amount of logic to be added to the memory system. Logic associated with memory is required for resolving conflict. The method often used to resolve conflicts is to assign permanently designated priorities to each memory port.

Non-uniform Memory Access(NUMA)

In NUMA architecture, all processors have access to all parts of main memory using loads and stores. The memory access time of a processor differs depending on which region of main memory is accessed. The last statement is true for all processors; however, for different processors, which memory regions are slower and which are faster differ.

A NUMA system in which cache coherence is maintained among the cache of the various processors is known as cache-cohence NUMA (CC-NUMA)

A typical CC-NUMA organization is shown in the Figure 10.4.

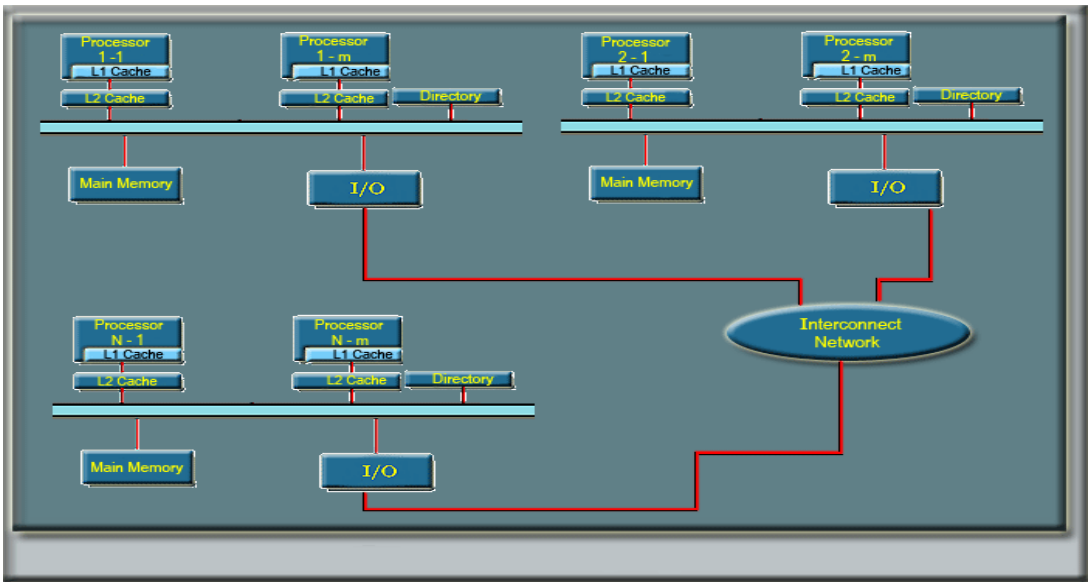


Figure 10.4: CC- NUMA Organization

There are multiple independent nodes, each of which is, in effect, an SMP organization.

Each node contains multiple processors, each with its own *L1* and *L2* caches, plus main memory.

The node is the basic building block of the overall CC NUMA organization

The nodes are interconnected by means of some communication facility, which could be a switching mechanism, a ring, or some other networking facility.

Each node in the CC-NUMA system includes some main memory.

From the point of view of the processors, there is only a single addressable memory, with each location having a unique system-wide address.

When a processor initiates a memory access, if the requested memory location is not in the processors cache, then the *L2* cache initiates a fetch operation.

If the desired line is in the local portion of the main memory, the line is fetch across the local bus.

If the desired line is in a remote portion of the main memory, then an automatic request is send out to fetch that line across the interconnection network, deliver it to the local bus, and then deliver it to the requesting cache on that bus.

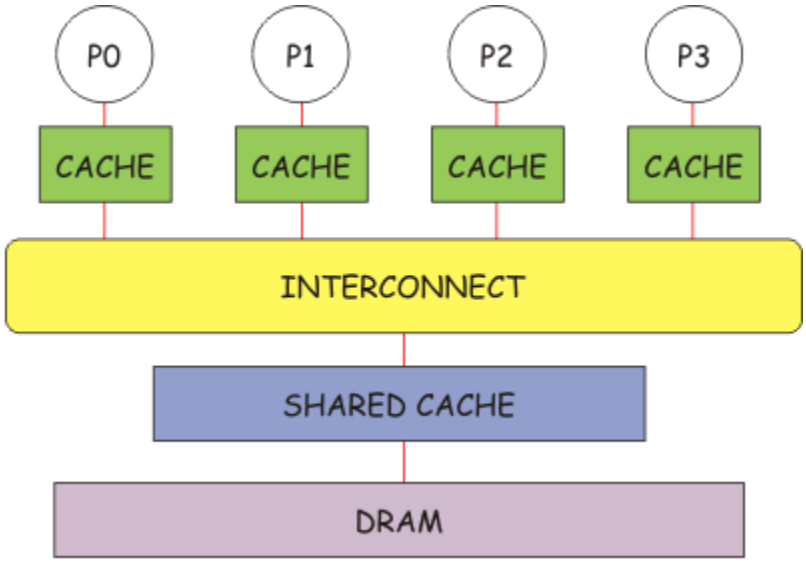
All of this activity is atomic and transparent to the processors and its cache.

In this configuration, cache coherence is a central concern. For that each node must maintain some sort of directory that gives it an indication of the location of various portion of memory and also cache status information.

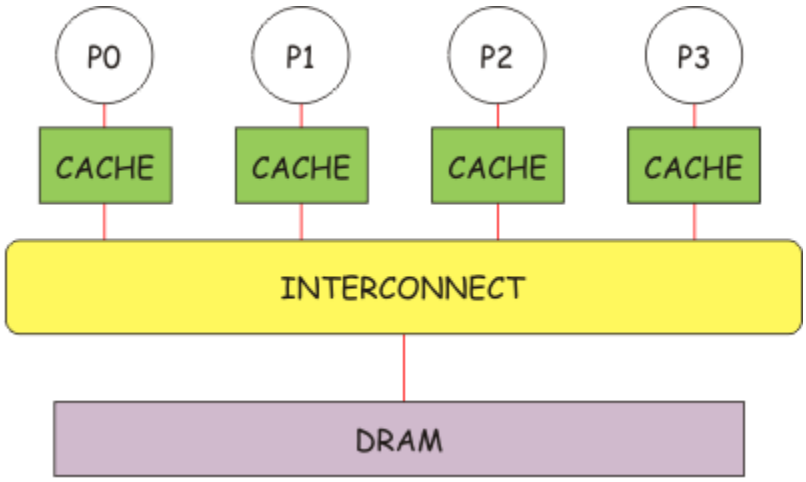
Shared memory multiprocessors

- What do they look like?
  - We will assume that each processor has a hierarchy of caches (possibly shared)
  - We will not discuss shared memory in a time-shared single-thread computer
- A degenerate case of the following
  - Shared cache (popular in CMPs)
  - Private cache (popular in CMPs and SMPs)
  - Dancehall (popular in old computers)
  - Distributed shared memory (popular in medium to large-scale servers)

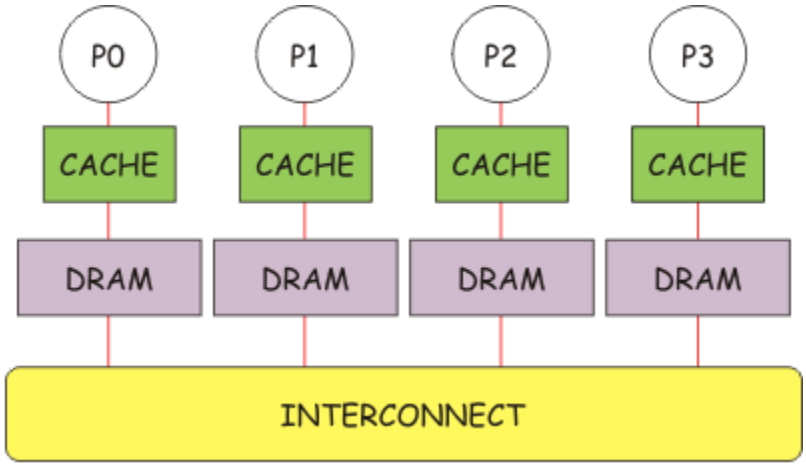
Shared cache



Private cache/Dancehall

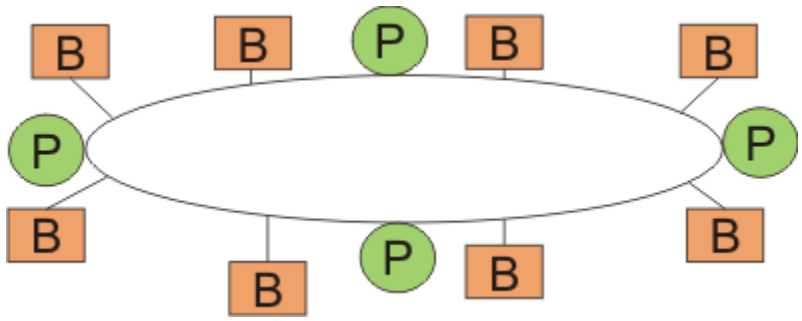


Distributed shared memory



Shared vs. private in CMPs

- Shared caches are often very large in the CMPs
  - They are banked to avoid worst-case wire delay
  - The banks are usually distributed across the floor of the chip on an interconnect



- In shared caches, getting a block from a remote bank takes time proportional to the physical distance between the requester and the bank
    - Non-uniform cache architecture (NUCA)
- This is same for private caches, if the data resides in a remote cache
- Shared cache may have higher average hit latency than the private cache
    - Hopefully most hits in the latter will be local
- Shared caches are most likely to have less misses than private caches
    - Latter wastes space due to replication

CACHE COHERENCE IN SHARED MEMORY

In contemporary multiprocessor system, it is customary to have one or two levels of cache associated with each processor. This organization is essential to achieve high performance.

Cache creates a problem, which is known as the cache coherence problem. **The cache coherence problem is: Multiple copies of the same data can exist in different caches simultaneously, and if processors are allowed to update their own copies freely, an inconsistent view of memory can result.**

There are two write policies:

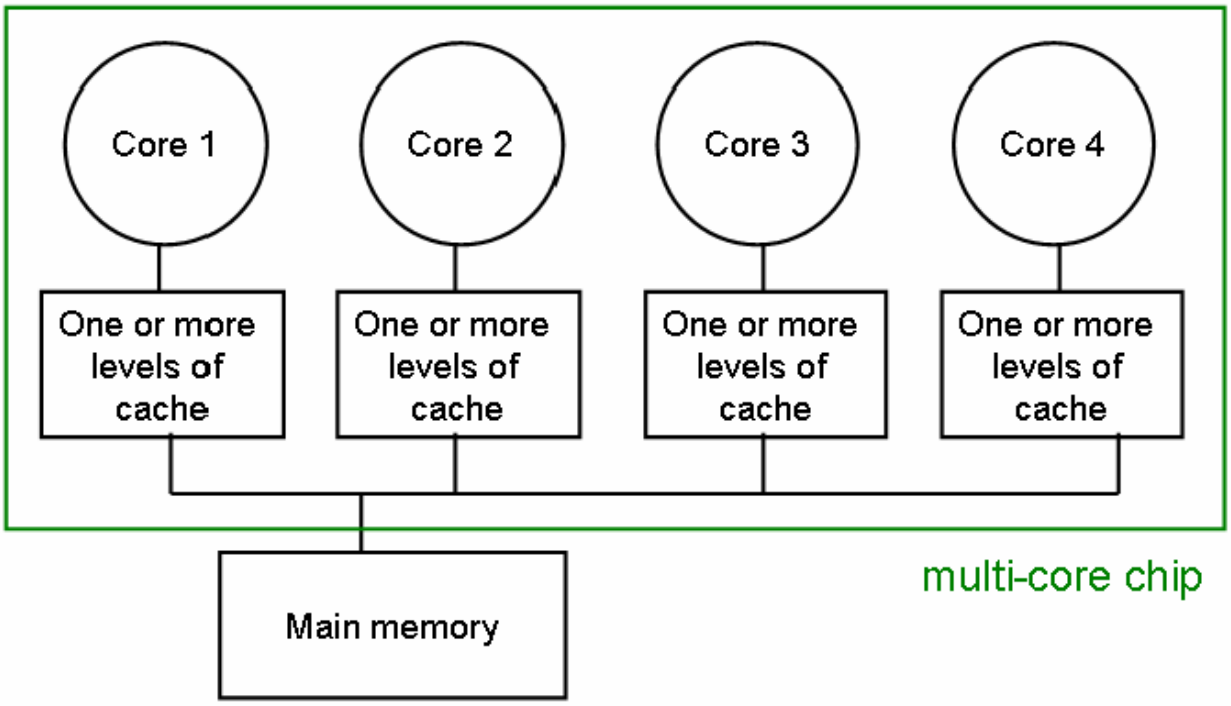
- **Write back:** Write operations are usually made only to the cache. Main memory is only updated when the corresponding cache line is flushed from the cache.
- **Write through:** All write operations are made to main memory as well as to the cache, ensuring that main memory is always valid.

It is clear that a write back policy can result in inconsistency.

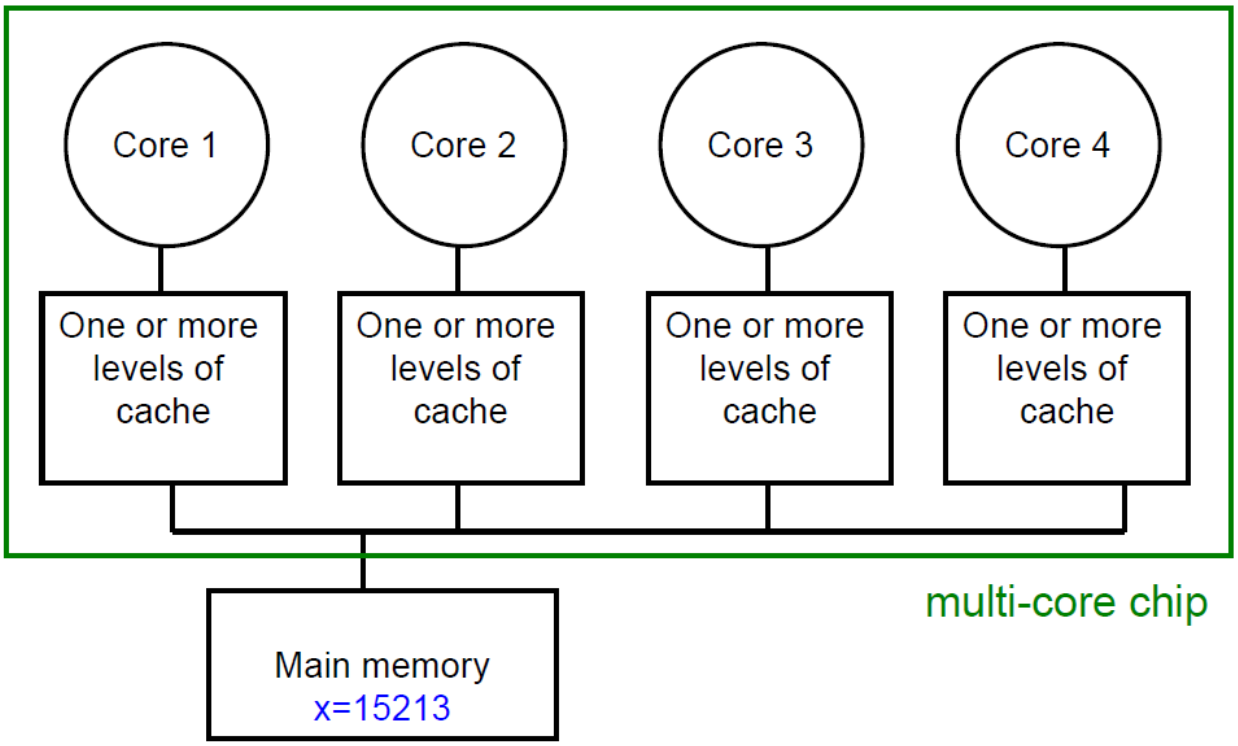
If two caches contain the same line, and the line is updated in one cache, the other cache will unknowingly have an invalid value. Subsequently read to that invalid line produce invalid results.

Even with the write through policy, inconsistency can occur unless other cache monitor the memory traffic or receive some direct notification of the update.

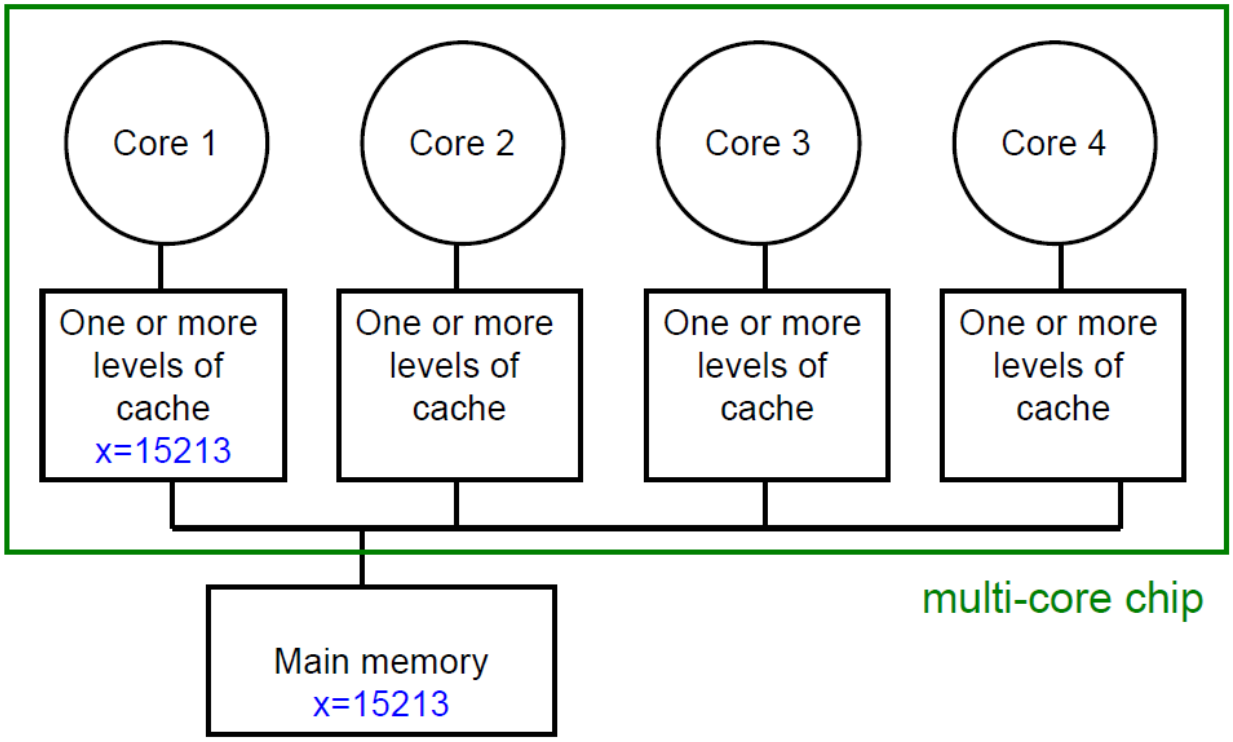
For any cache coherence protocol, the objective is to let recently used local variables get into the appropriate cache and stay there through numerous reads and write, while using the protocol to maintain consistency of shared variables that might be in multiple caches at the same time.



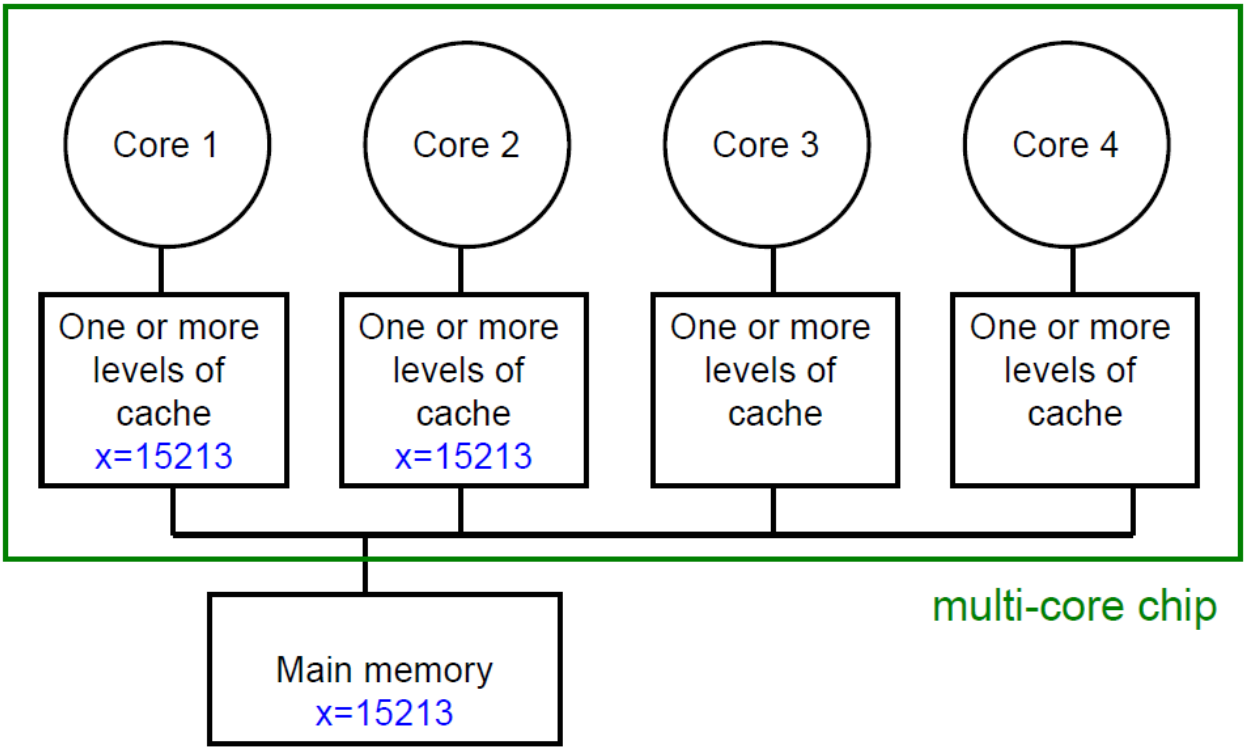
Suppose variable x initially contains 15213



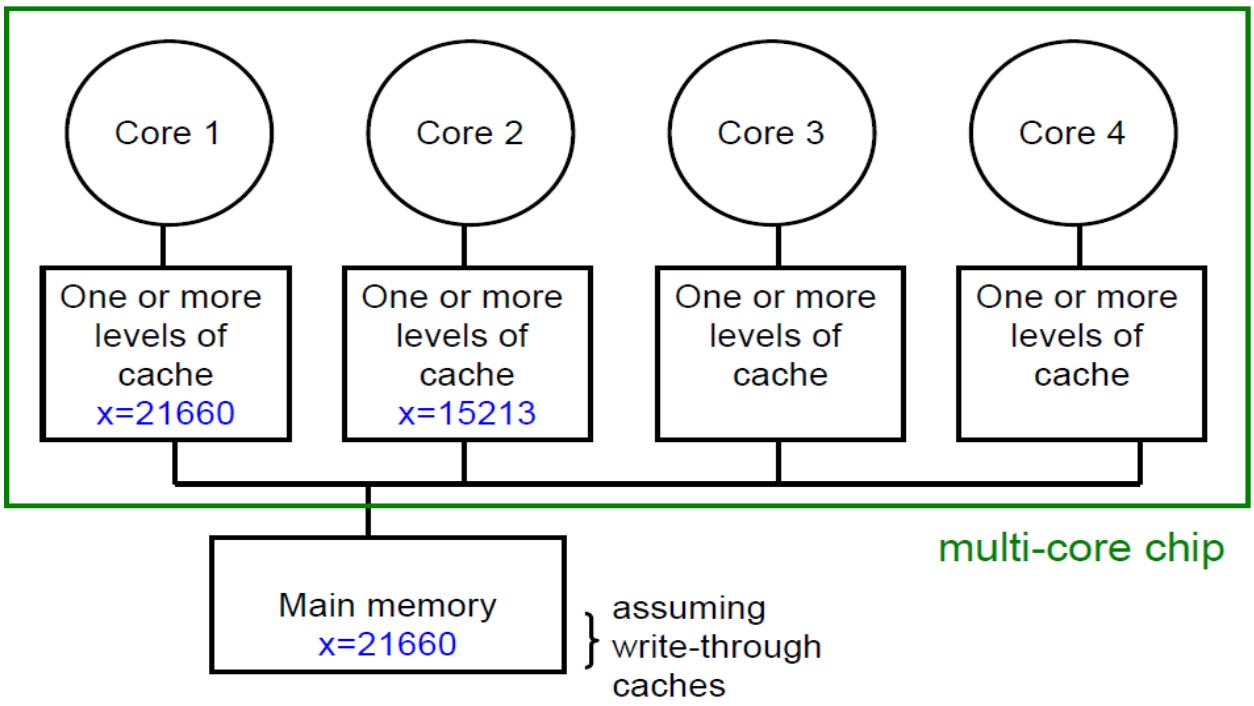
Core 1 reads x



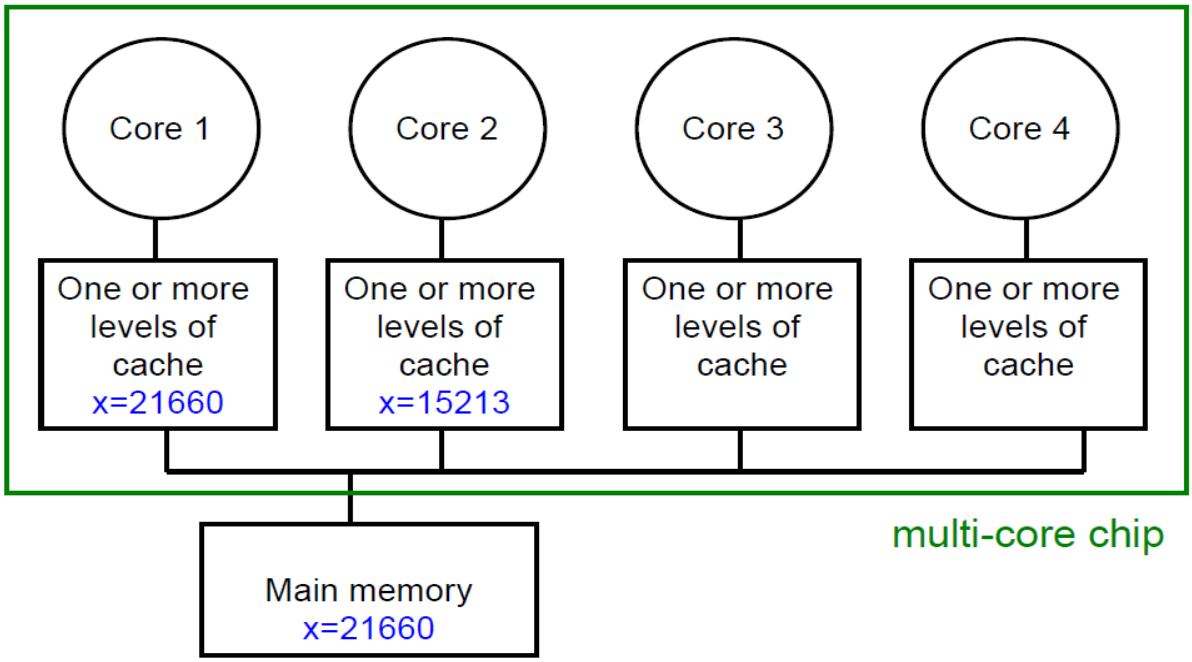
Core 2 reads x



Core 1 writes to x, setting it to 21660



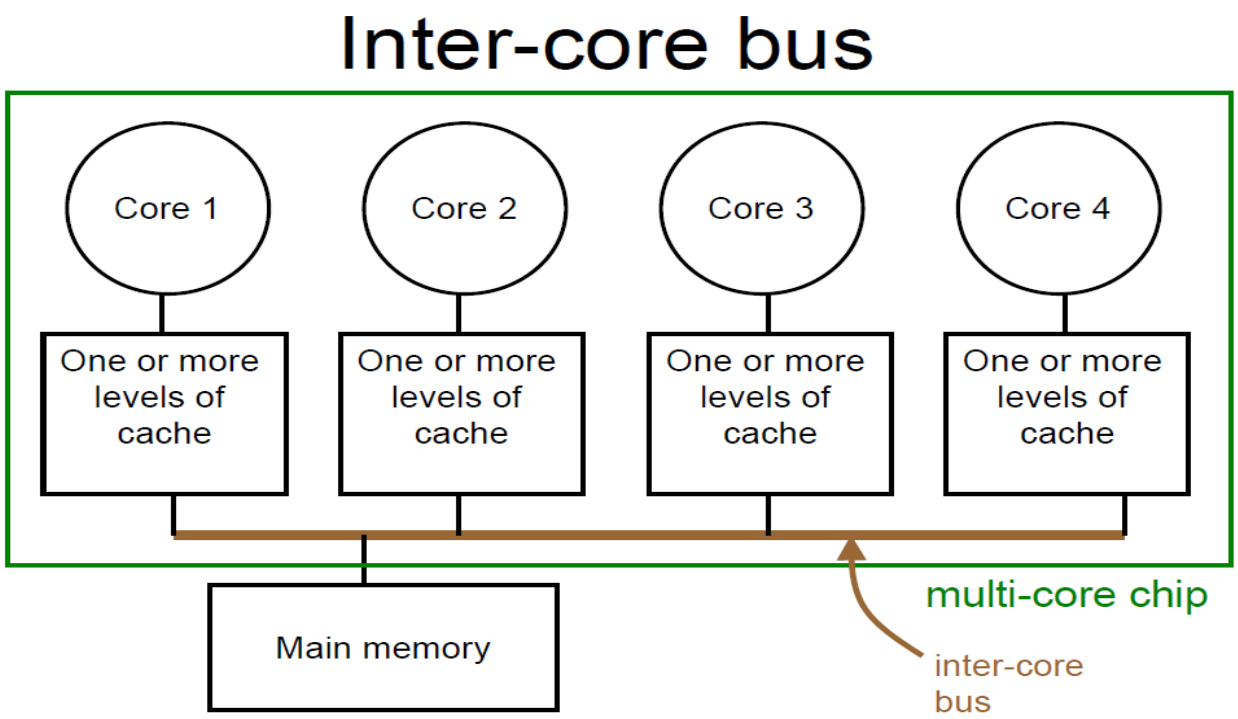
Core 2 attempts to read x… gets a stale copy



Solutions for cache coherence

- This is a general problem with multiprocessors, not limited just to multi-core

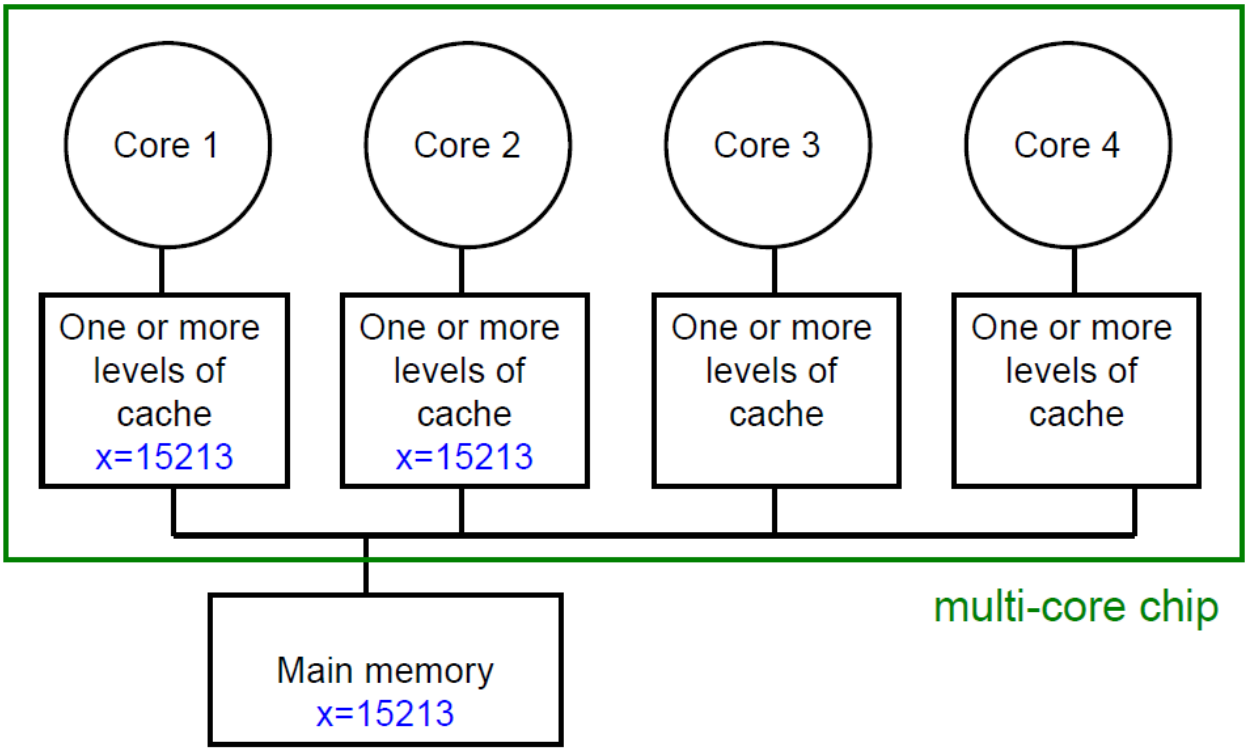
- There exist many solution algorithms, coherence protocols, etc.
- A simple solution: invalidation-based protocol with snooping



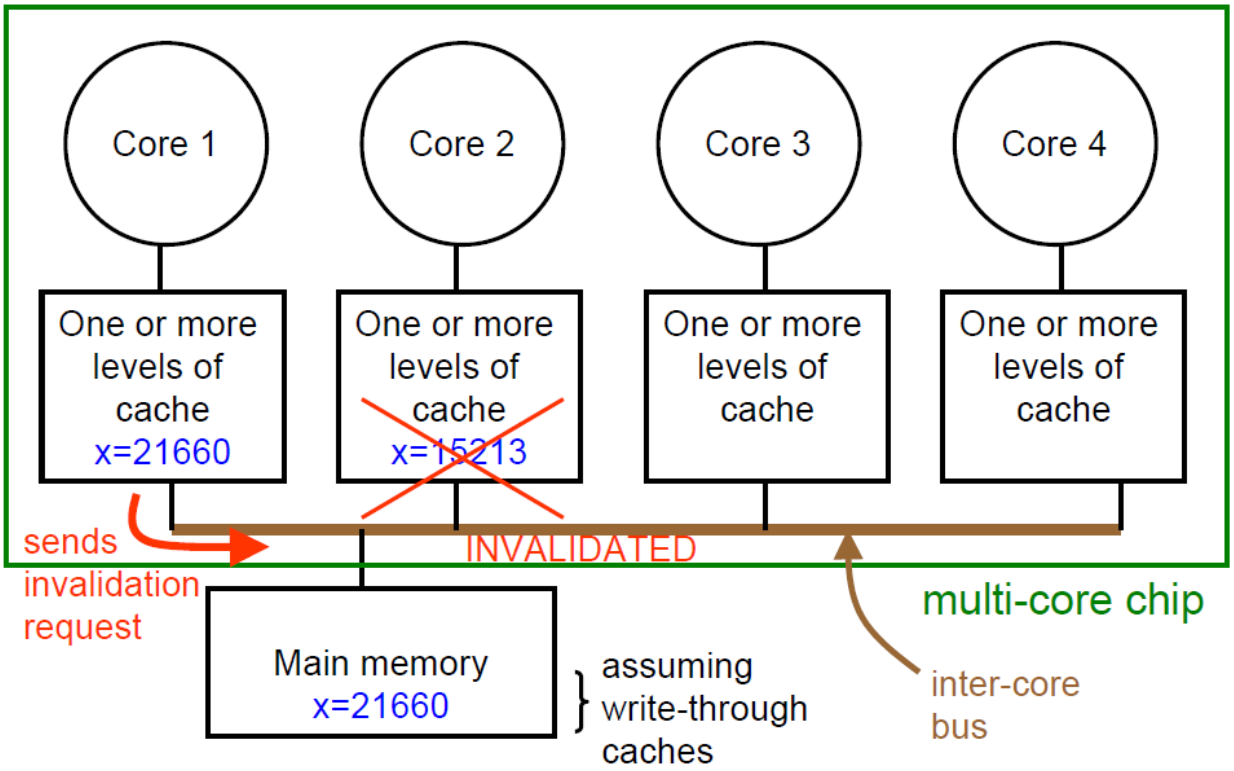
Invalidation protocol with snooping

- Invalidation:
  - If a core writes to a data item, all other copies of this data item in other caches are invalidated
- Snooping:
  - All cores continuously "snoop" (monitor) the bus connecting the cores.

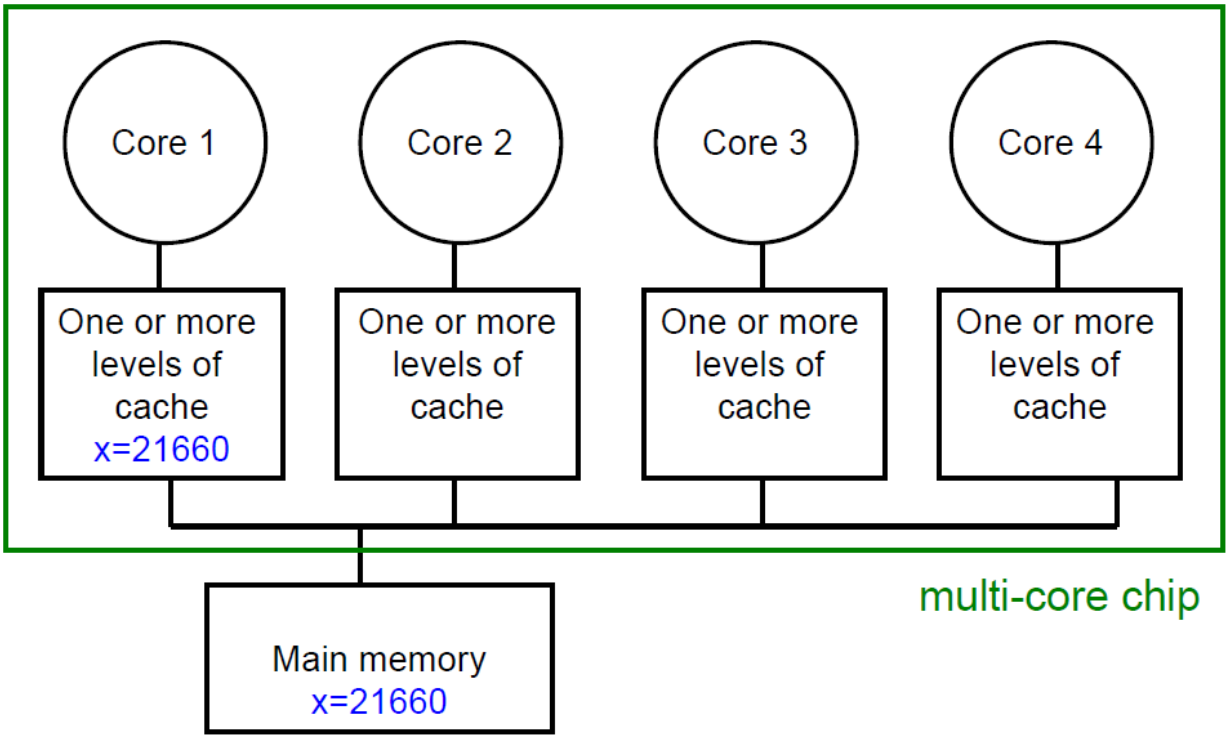
Revisited: Cores 1 and 2 have both read x



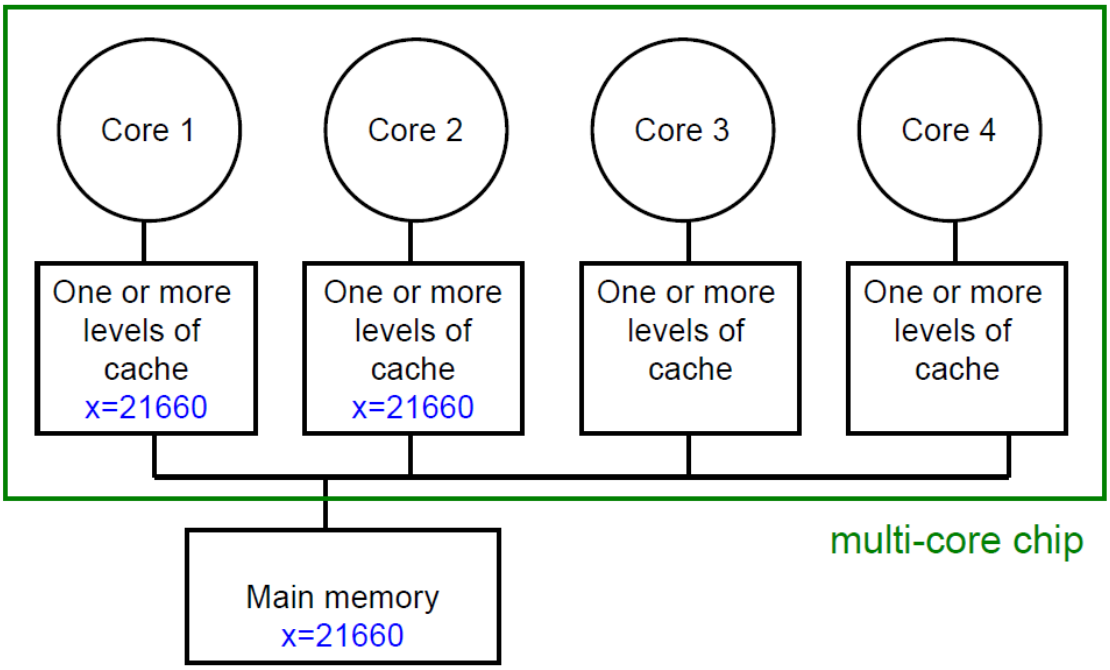
Core 1 writes to x, setting it to 21660



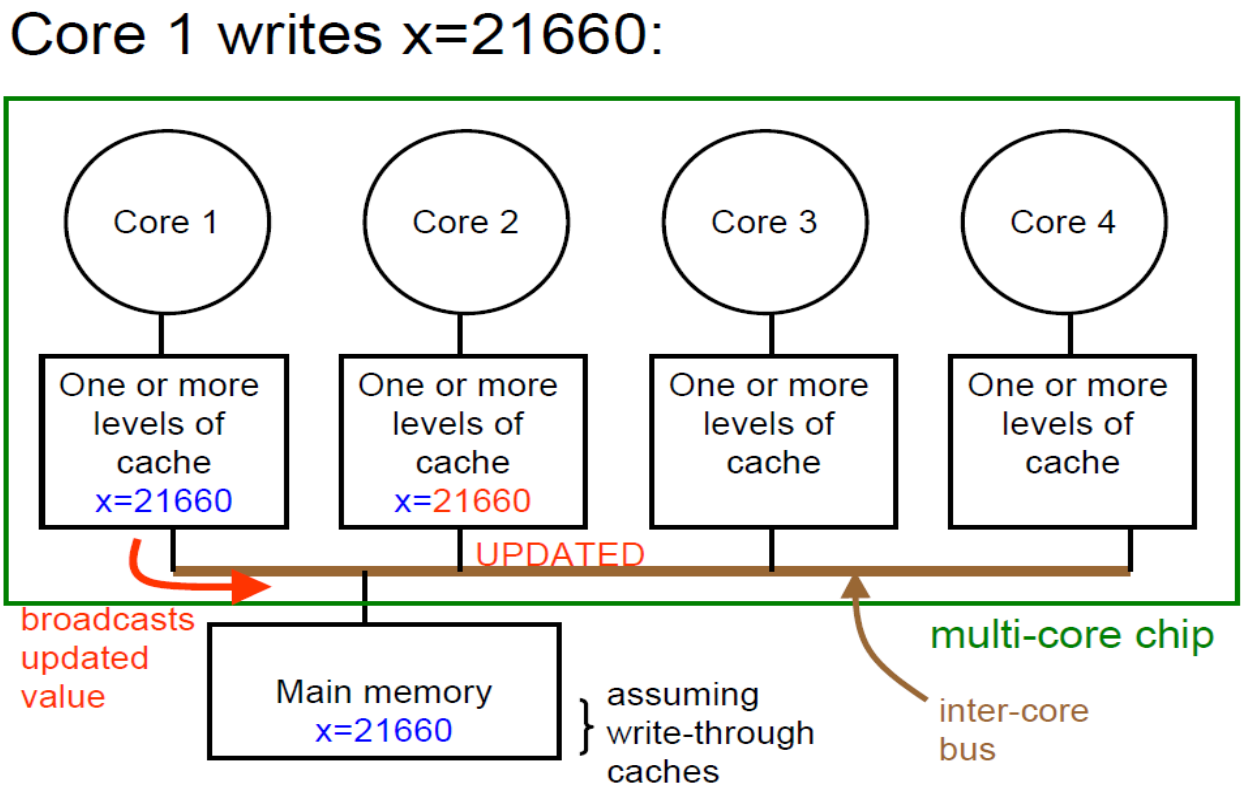
After invalidation:



Core 2 reads x. Cache misses, and loads the new copy.



Alternative to invalidate protocol: update protocol



Which do you think is better Invalidation or update?

Invalidation vs. update

- Multiple writes to the same location
  - invalidation: only the first time
  - update: must broadcast each write (which includes new variable value)
- Invalidation generally performs better:
  - it generates less bus traffic

The individual processor reads from its cache memory. The individual processor writes data to its cache memory.

The individual processor must write back a dirty line from its

cache in order to free up a cache line for a newly requested block.

Another processor issues a read request to the shared primary memory for a block that is held in this processors individual cache. This processor's snoop cache detects the request.

Another processor issues a write request to the shared primary

memory for a block that is held in this processors individual cache.

Another processor signals that a write to a cache line that is shared with this processor. The other processor will upgrade the status of the cache line from “Shared” to “Modified”.

Write through protocol:

A write through protocol can be implemented in two fundamental versions.

(a) Write through with update protocol (b) Write through with invalidation of copies

Write through with update protocol:

When a processor writes a new value into its cache, the new value is also written into the memory module that holds the cache block being changed. Some copies of this block may exist in other caches, these copies must be updated to reflect the change caused by the write operation.

The simplest way of doing this is to broadcast the written data to all processor modules in the system.

As each processor module receives the broadcast data, it updates the contents of the affected cache block if this block is present in its cache.

Write through with invalidation of copies:

When a processor writes a new value into its cache, this value is written into the memory module, and all copies in the other caches are invalidated. Again broadcasting can be used to send the invalidation requests through the system.

Write back protocol:

In the write-back protocol, multiple copies of a cache block may exist if different processors have loaded (read) the block into their caches.

If some processor wants to change this block, it must first become an exclusive owner of this block.

When the ownership is granted to this processor by the memory module that is the home location of the block. all other copies, including the one in the memory module, are invalidated.

Now the owner of the block may change the contents of the memory.

When another processor wishes to read this block, the data are sent to this processor by the current owner.

The data are also sent to the home memory module, which requires ownership and updates the block to contain the latest value.

There are software and hardware solutions for cache coherence problem.

Software solution:

In software approach, the detecting of potential cache coherence problem is transferred from run time to compile time, and the design complexity is transferred from hardware to software

Compiler-based cache coherence mechanism performs an analysis on the code to determine which data items may become unsafe for caching, and they mark those items accordingly. So, there are some non-cacheable items, and the operating system or hardware does not cache those items.

Hardware solution:

Hardware solution provides dynamic recognition at run time of potential inconsistency conditions. Because the problem is only dealt with when it actually arises, there is more effective use of caches, leading to improved performances over a software approach.

Hardware schemes can be divided into two categories: **(a) directory protocol** and **(b) snoopy protocols**.

(a) Directory protocols:

Directory protocols collect and maintain information about where copies of lines reside. Typically, there is a centralized controller that is part of the main memory controller, and a directory that is stored in main memory.

The directory contains global state information about the contents of the various local caches.

When an individual cache controller makes a request, the centralized controller checks and issues necessary commands for data transfer between memory and caches or between caches themselves.

It is also responsible for keeping the state information up to date, therefore, every local action that can affect the global state of a line must be reported to the central controller.

The controller maintains information about which processors have a copy of which lines.

Before a processor can write to a local copy of a line, it must request exclusive access to the line from the controller.

Before granting this exclusive access, the controller sends a message to all processors with a cached copy of this time, forcing each processors to invalidate its copy.

After receiving acknowledgement back from each such processor, the controller grants exclusive access to the requesting processor.

When another processor tries to read a line that is exclusively granted to another processors, it will send a miss notification to the controller.

The controller then issues a command to the processor holding that line that requires the processors to do a write back to main memory.

Directory schemes suffer from the drawbacks of a **central bottleneck** and the **overhead of communication** between the various cache controllers and the central controller.

(b) Snoopy protocols:

Snoopy protocols distribute the responsibility for maintaining cache coherence among all of the cache controllers in a multiprocessor system.

A cache must recognize when a line that it holds is shared with other caches.

When an update action is performed on a shared cache line, it must be announced to all other caches by a broadcast mechanism.

Each cache controller is able to "snoop" on the network to observed these broadcasted notifications and react accordingly.

Snoopy protocols are ideally suited to a bus-based multiprocessor, because the shared bus provides a simple means for broadcasting and snooping.

Two basic approaches to the snoopy protocol have been explored: **Write invalidates** or **write-update (write-broadcast).**

(i) Write invalidates:

With a write-invalidate protocol, there can be multiple readers but only one write at a time.

Initially, a line may be shared among several caches for reading purposes.

When one of the caches wants to perform a write to the line it first issues a notice which invalidates the line in the other caches, making the line exclusive to the writing cache.

Once the line is exclusive, the owning processor can make local writes until some other processor requires the same line.

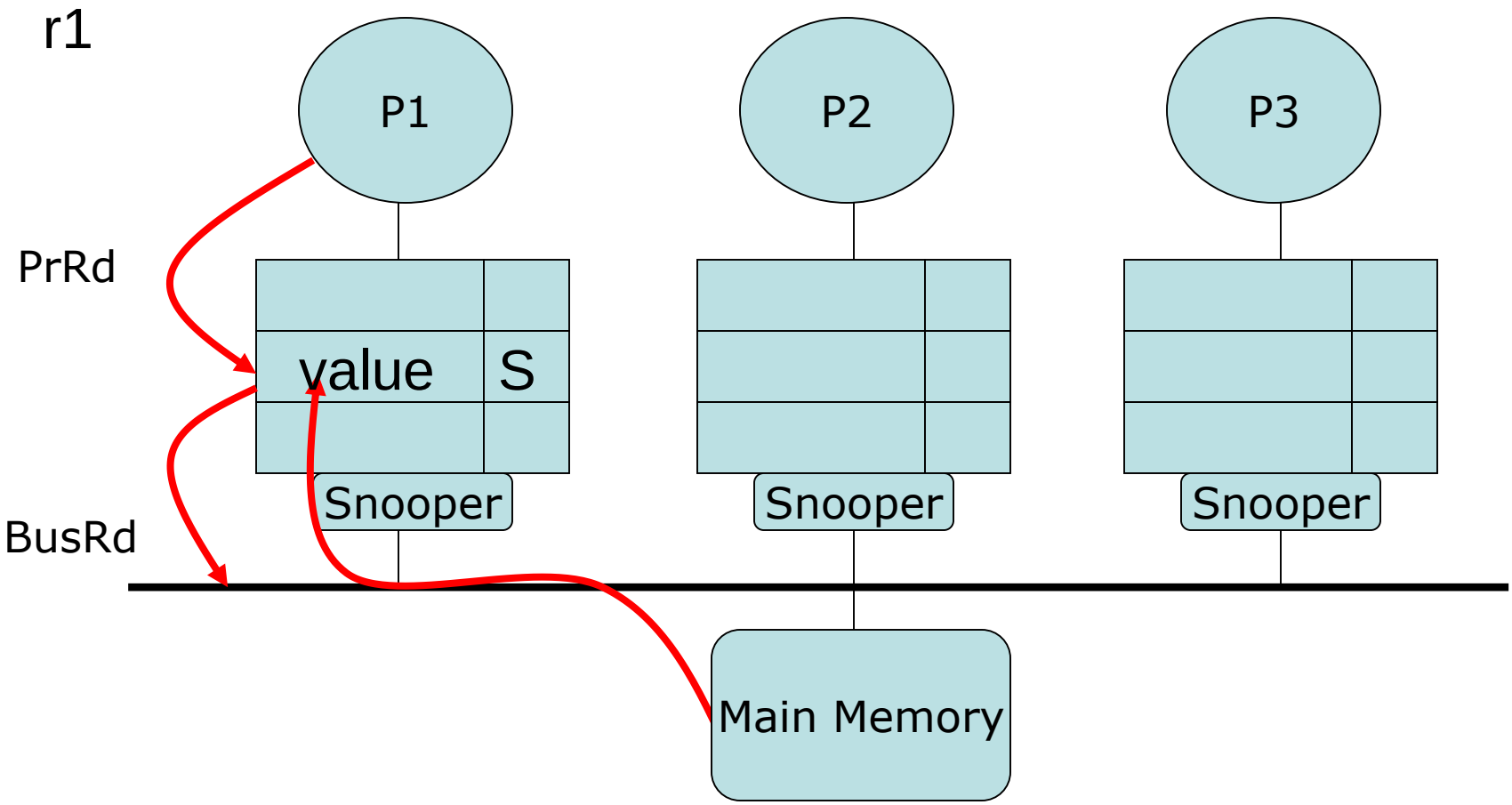
(ii) Write update (write broadcast):

With a write update protocol, there can be multiple writers as well as multiple readers. When a processor wishes to update a shared line, the word to be updated is distributed to all others, and caches containing that line can update it.

MSI Protocol

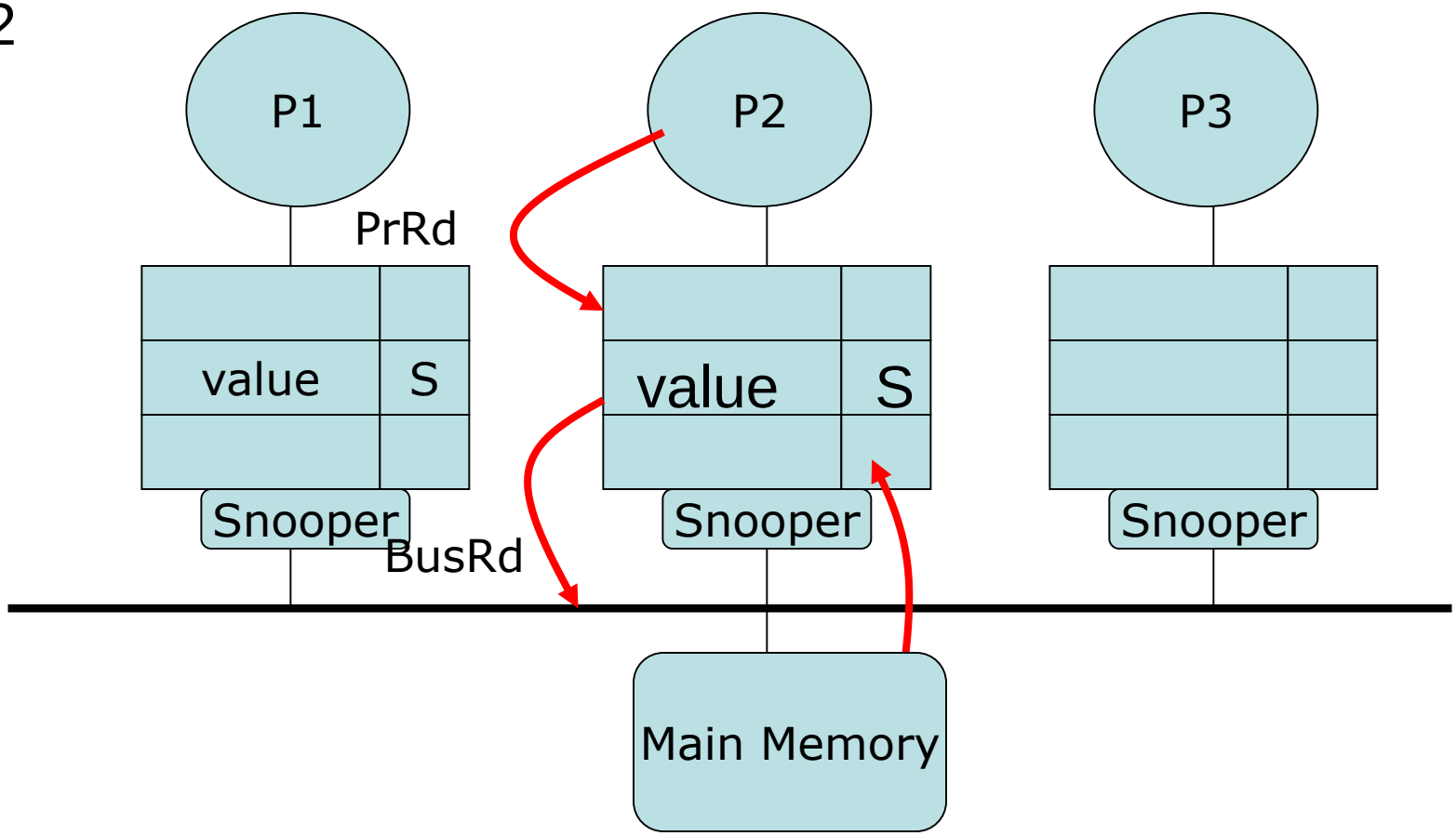
There are three processors. Each is reading/writing the same value from memory where r1 means a read by processor 1 and w3 means a write by processor 3. For simplicity sake, the memory location will be referred to as “value.” The memory access stream is:

r1, r2, w3, r2, w1, w2, r3, r2, r1



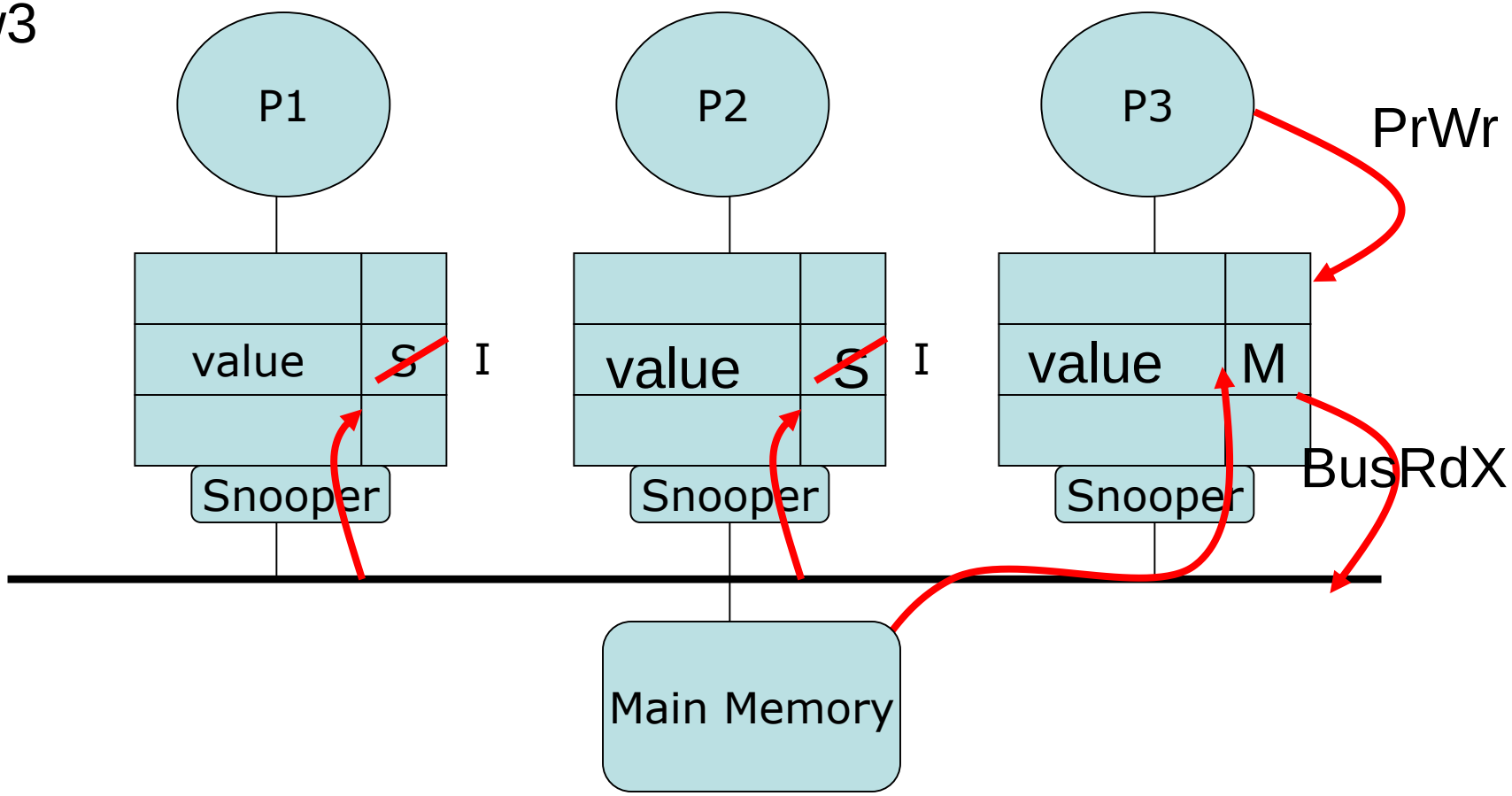
P1 wants to read the value. The cache does not have itand generates a BusRd for the data. Main memory controller provides the data. The data goes into the cache in the shared state.

r2



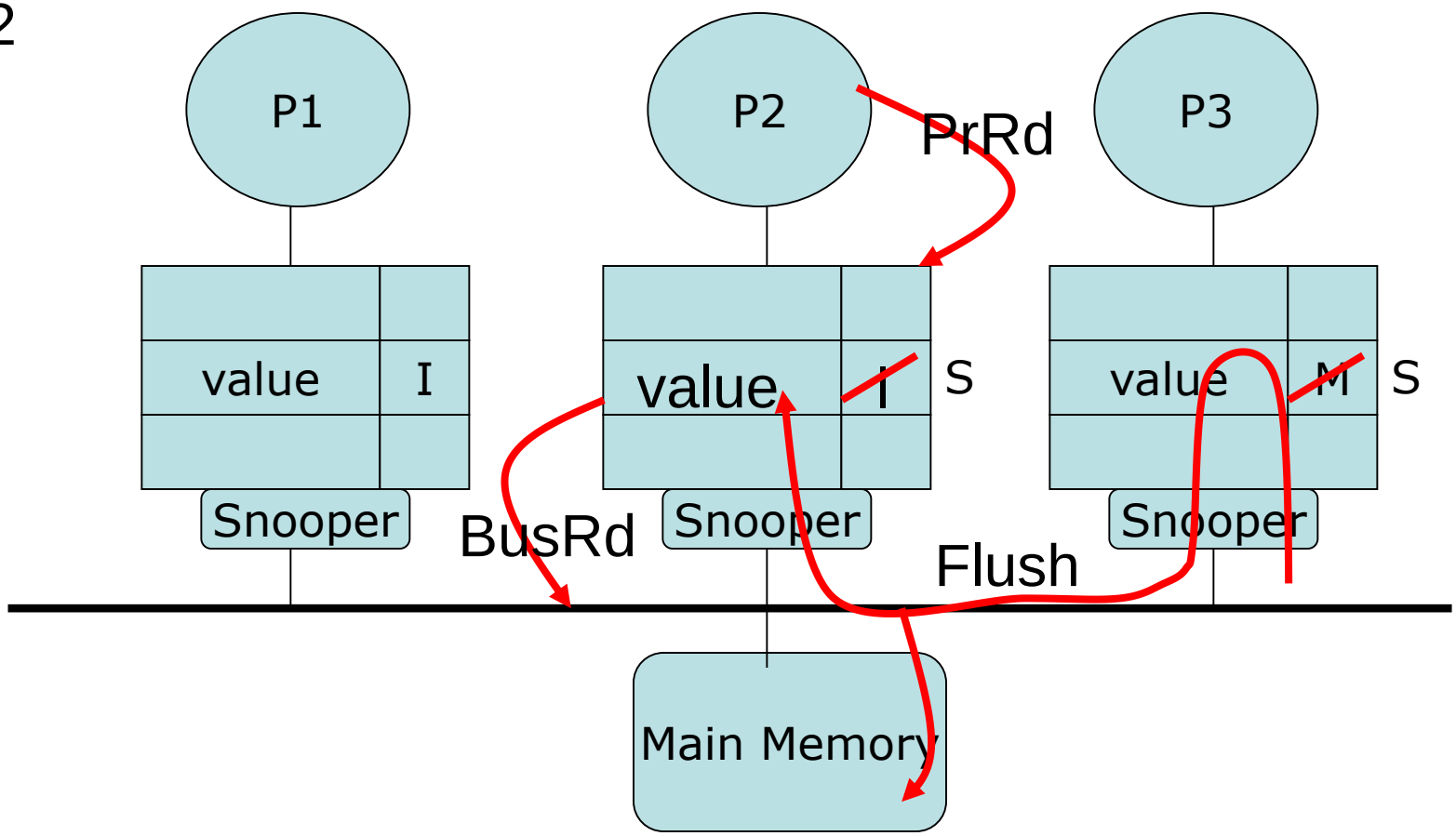
P2 wants to read the value. Its cache does not have the data, so it places a BusRd to notify other processors and ask for the data. The memory controller provides the data.

w3

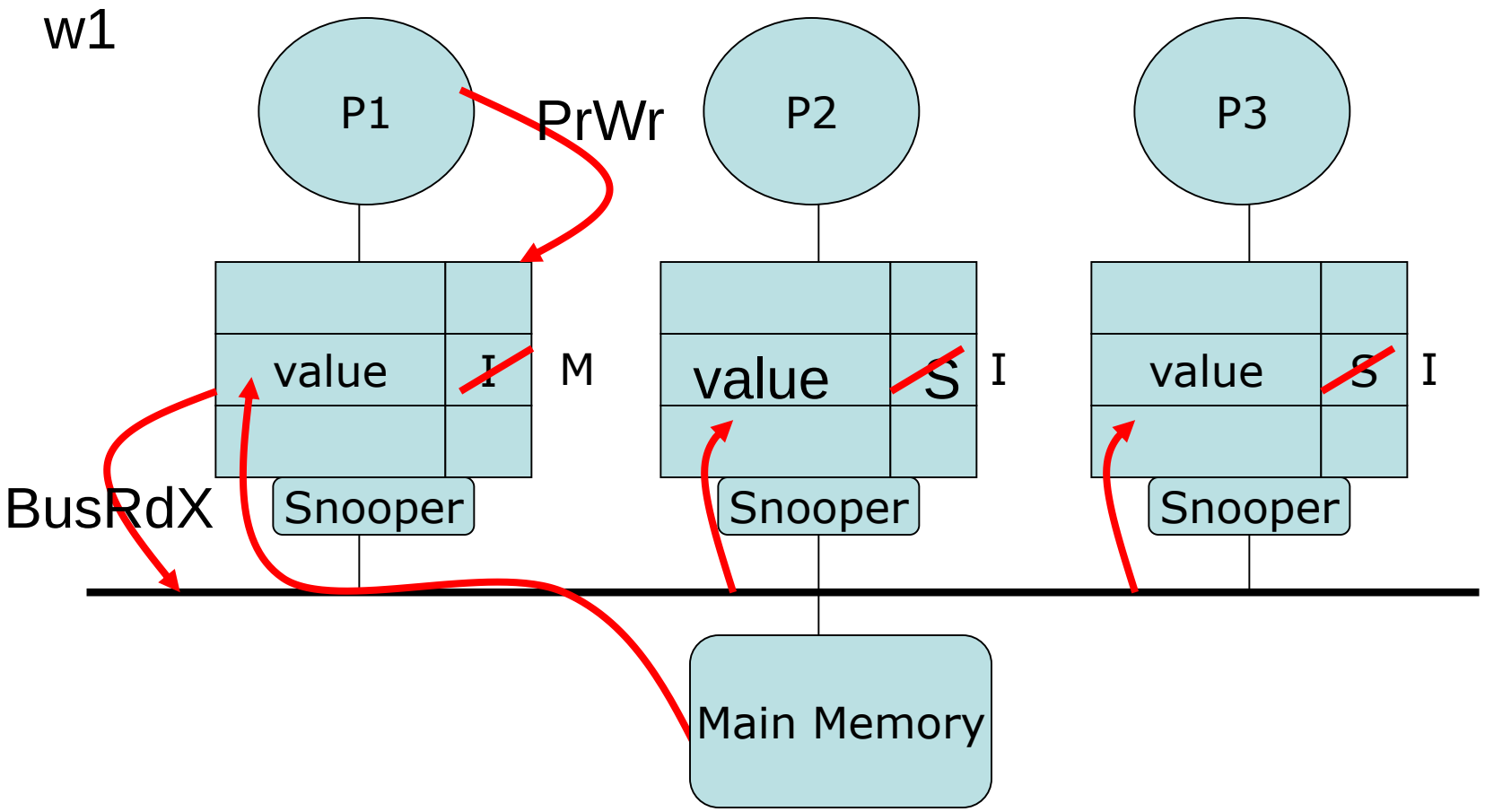


P3 wants to write the value. It places a BusRdX to get exclusive access and the most recent copy of the data. The caches of P1 and P2 see the BusRdX and invalidate their copies. Because the value is still up-to-date in memory, memory provides the data.

r2

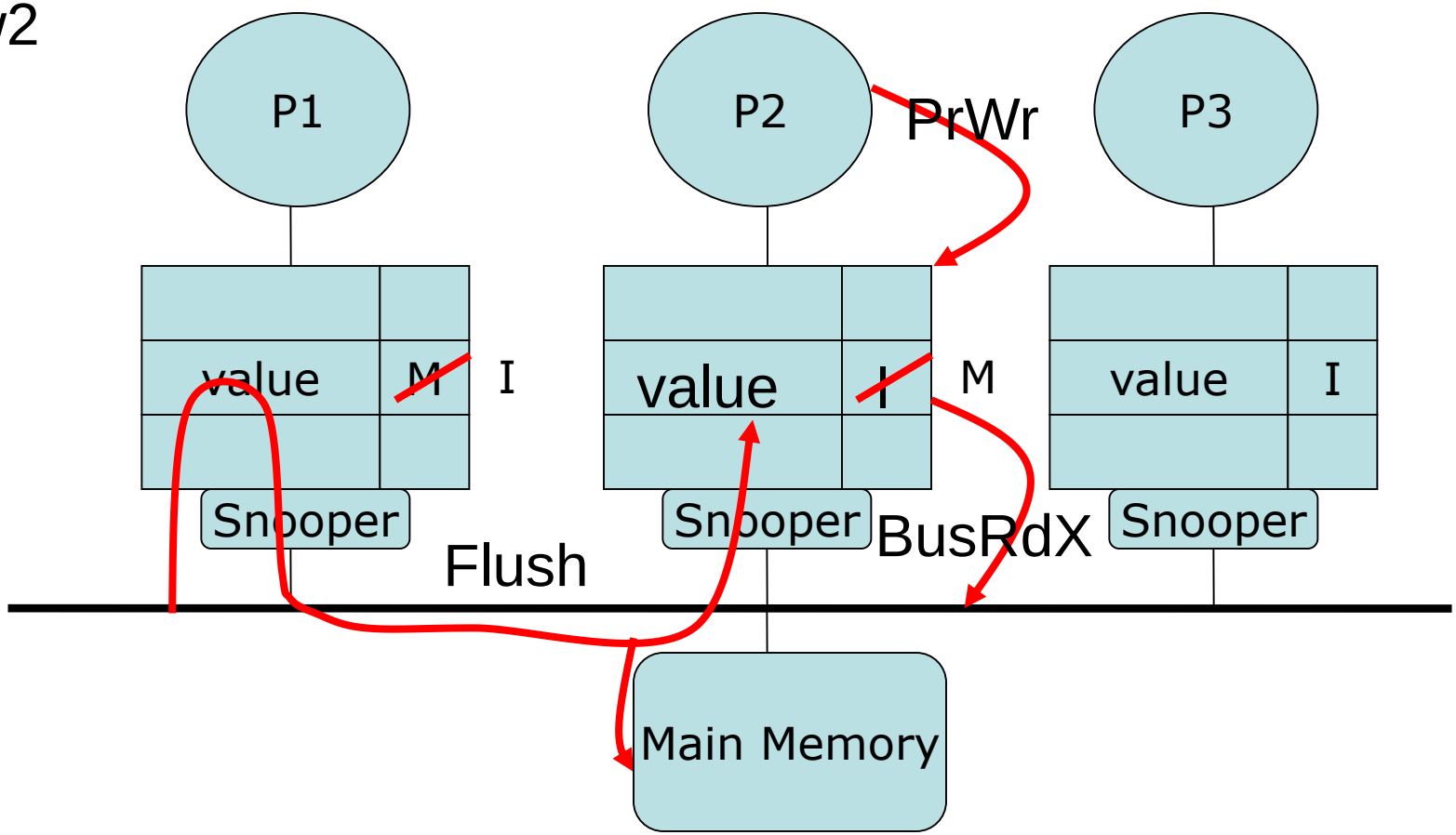


P2 wants to read the value. P3's cache has the most up-to-date copy and will provide it. P2's cache puts a BusRd on the bus. P3's cache snoops this and cancels the memory access because it will provide the data. P3's cache flushes the data to the bus.



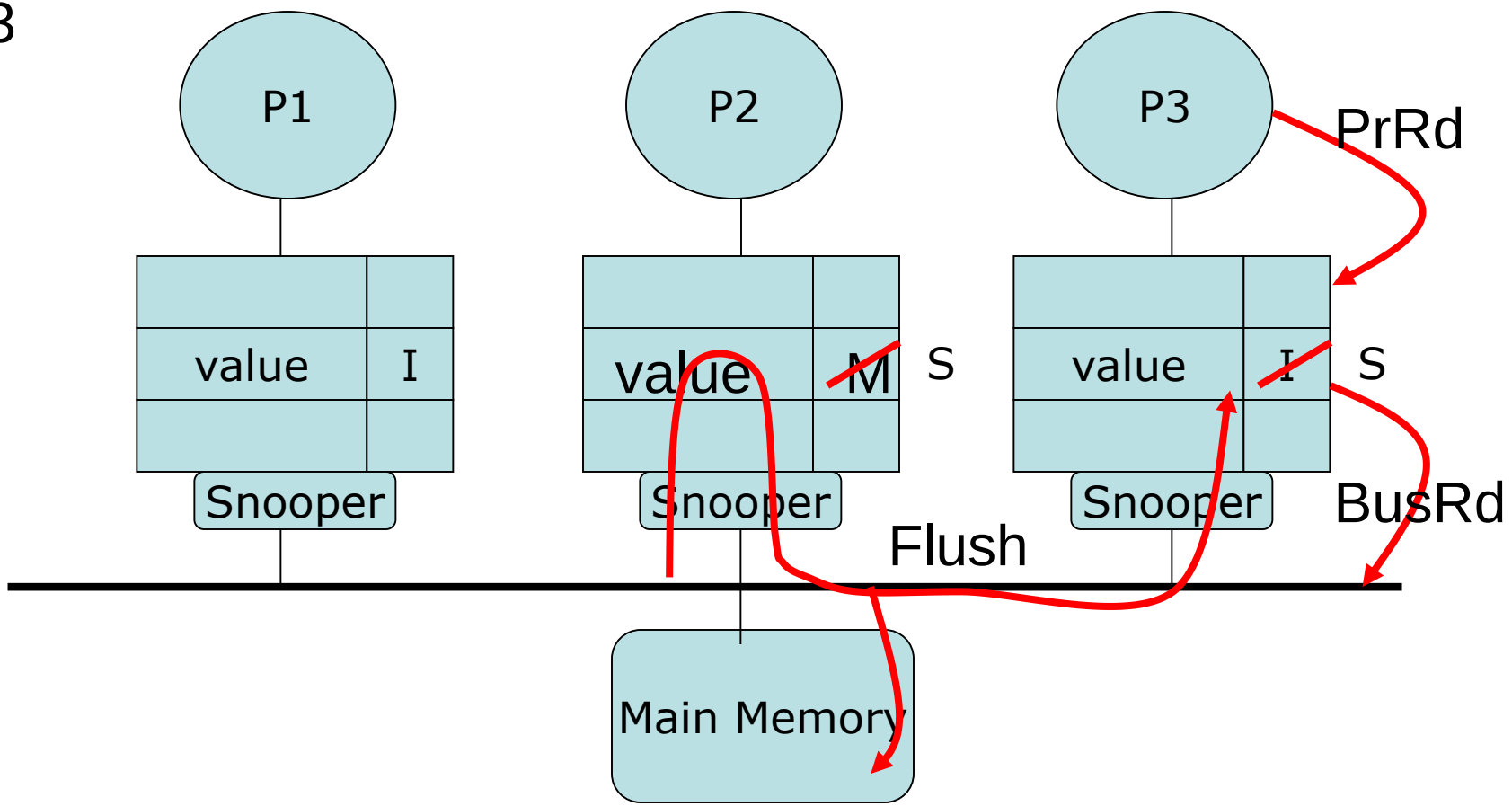
P1 wants to write to its cache. The cache places a BusRdX on the bus to gain exclusive access and the most up-to-date value. Main memory is not stale so it provides the data. The snoopers for P2 and P3 see the BusRdX and invalidate their copies in cache.

w2



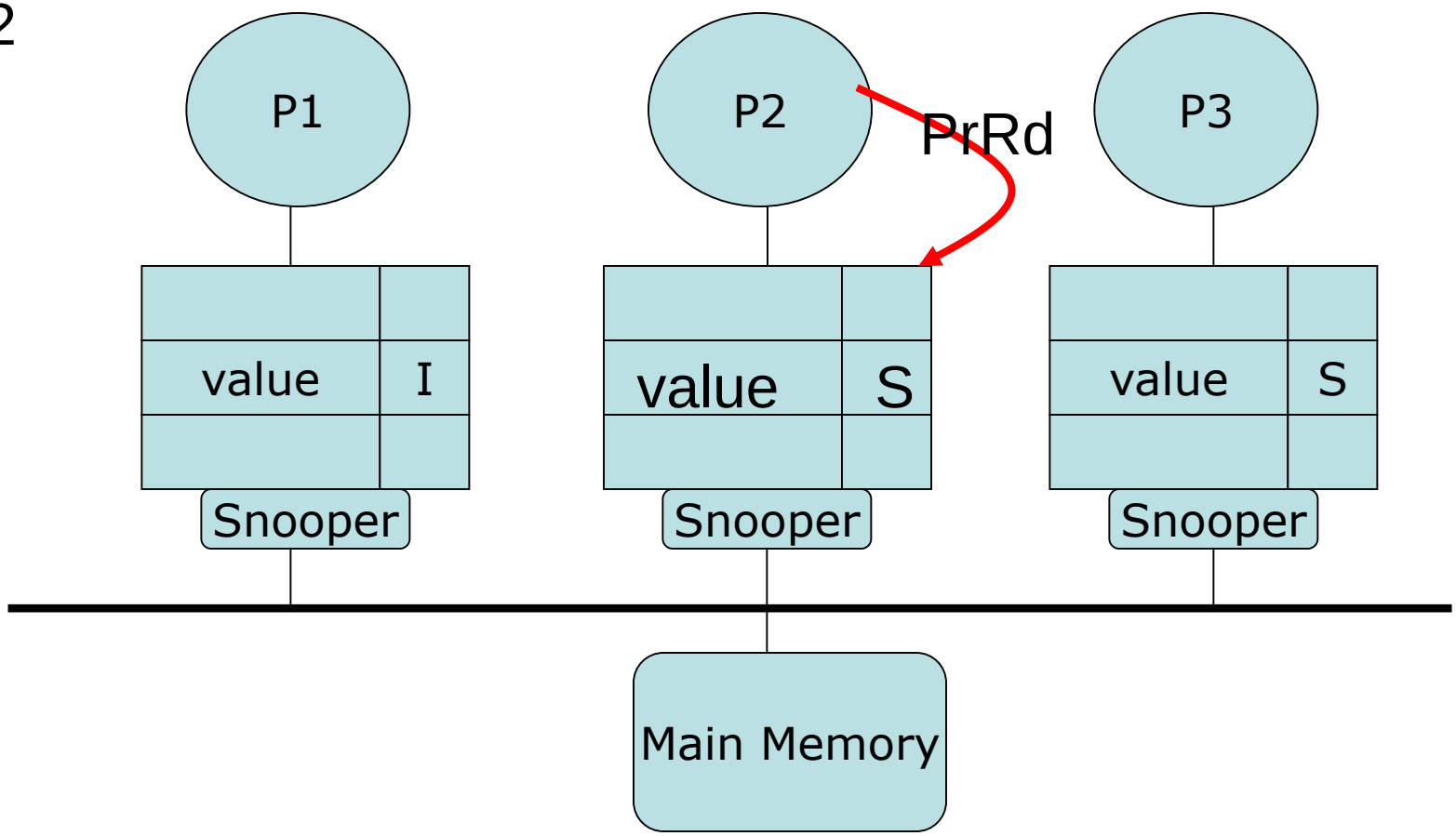
P2 wants to write the value. Its cache places a BusRdX to get exclusive access and the most recent copy of the data. P1's snooper sees the BusRdX and flushes the data to the bus. Also, it invalides the data in its cache and cancels the memory access.

r3

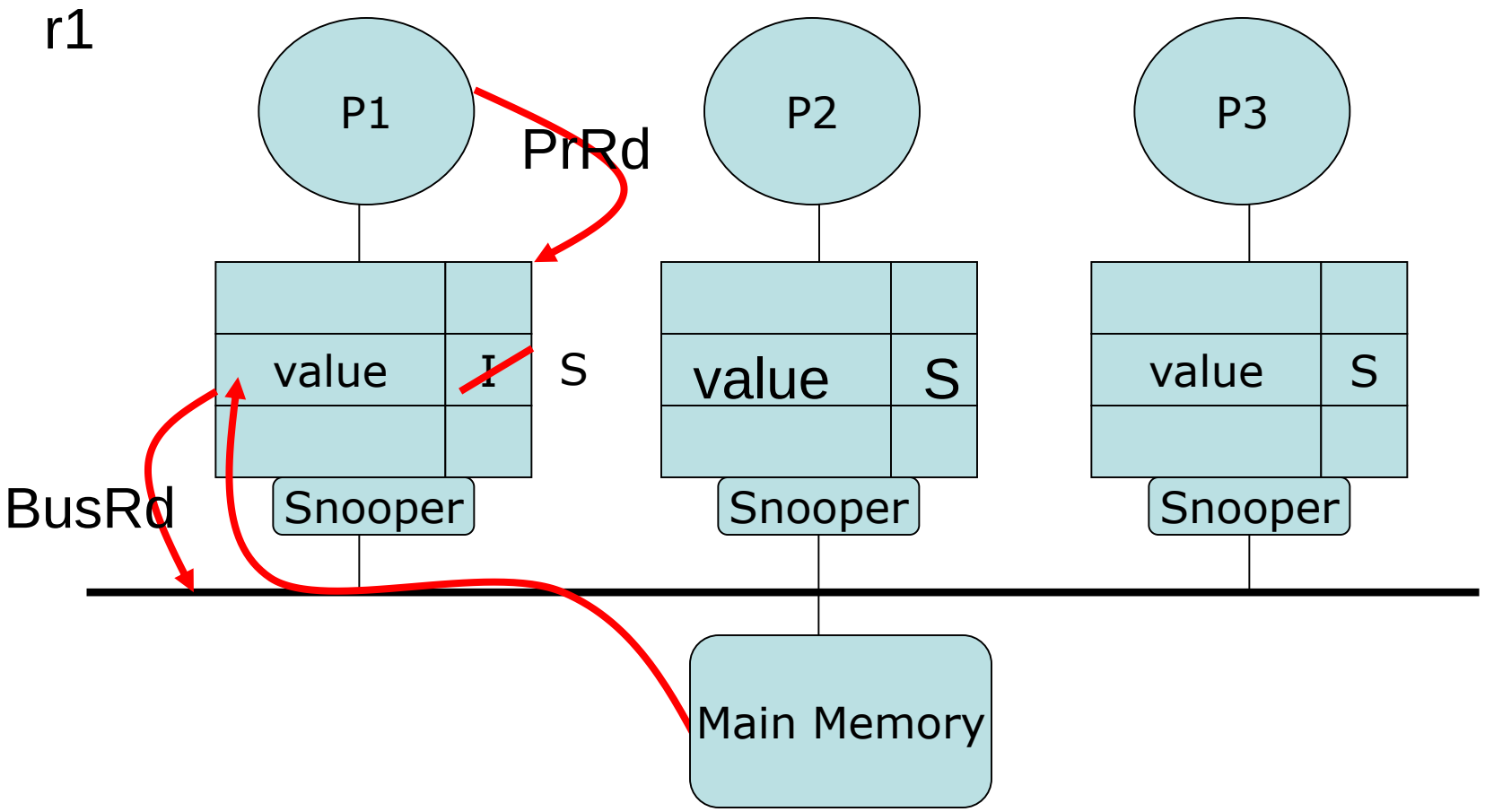


P3 wants to read the value. Its cache does not have a valid copy, so it places a BusRd on the bus. P2 has a modified copy, so it flushes the data on the bus and changes the status of the cache data to shared. The flush cancels the memory accecss and updates the data in memory as well.

r2



P2 wants to read the value. Its cache has an up-to-date copy. No bus transactions need to take place as there is no cache miss.



P1 wants to read the value. The cache does not have it, so it places a BusRd onto the bus for the data. The memory controller provides the data as it has an up-to-date copy. The data goes into the cache in the shared state.

MESI Protocol

There are three processors. Each is loading or storing different words from memory given as w0, w1, and w2. These all map to the same location in cache.

The memory accesses are as follows:

P1: ld w0, P3: ld w2

P1: st w0, P2: st w2

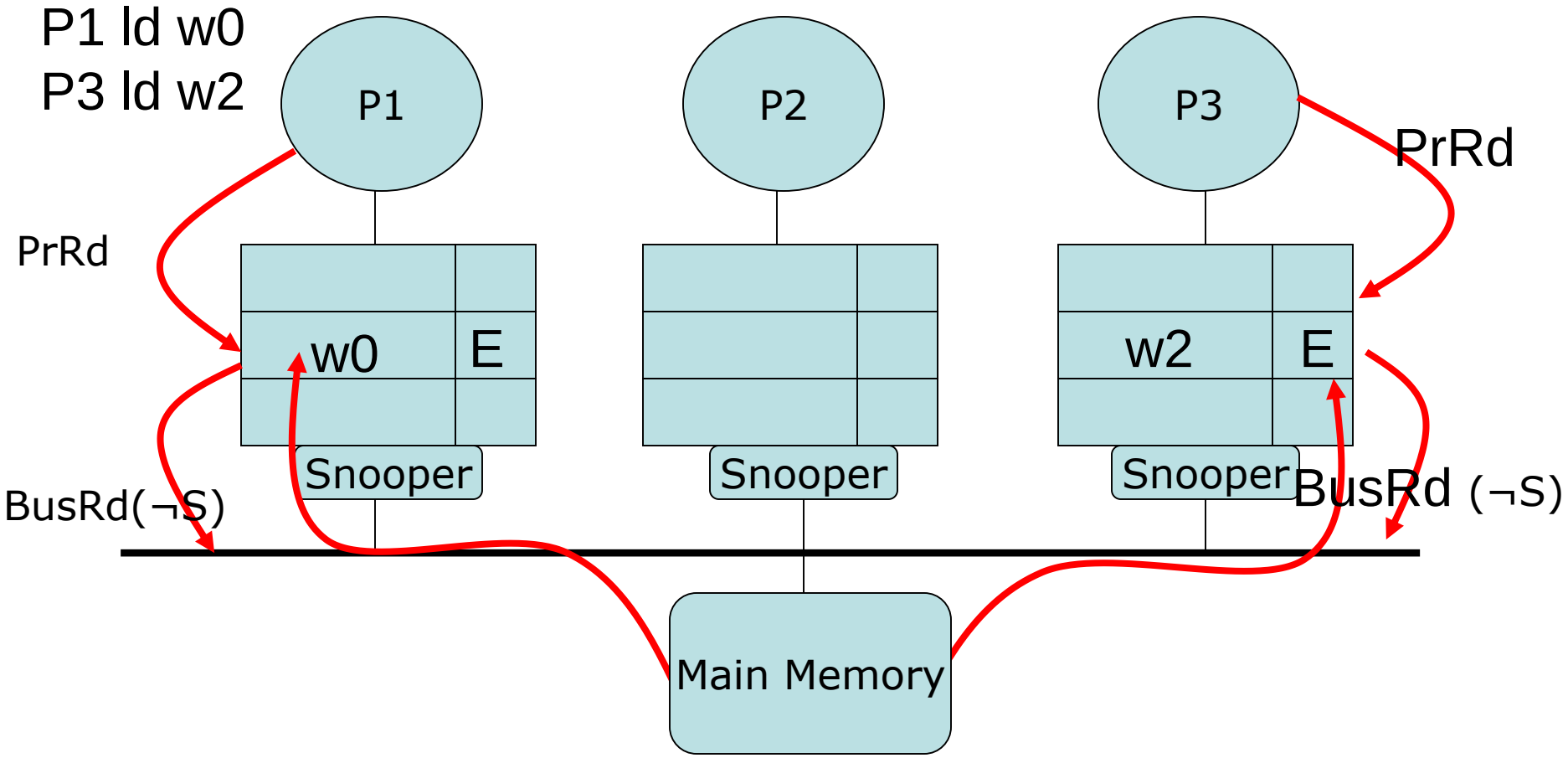
P2 st w2, P3 ld w0

P3: st w0

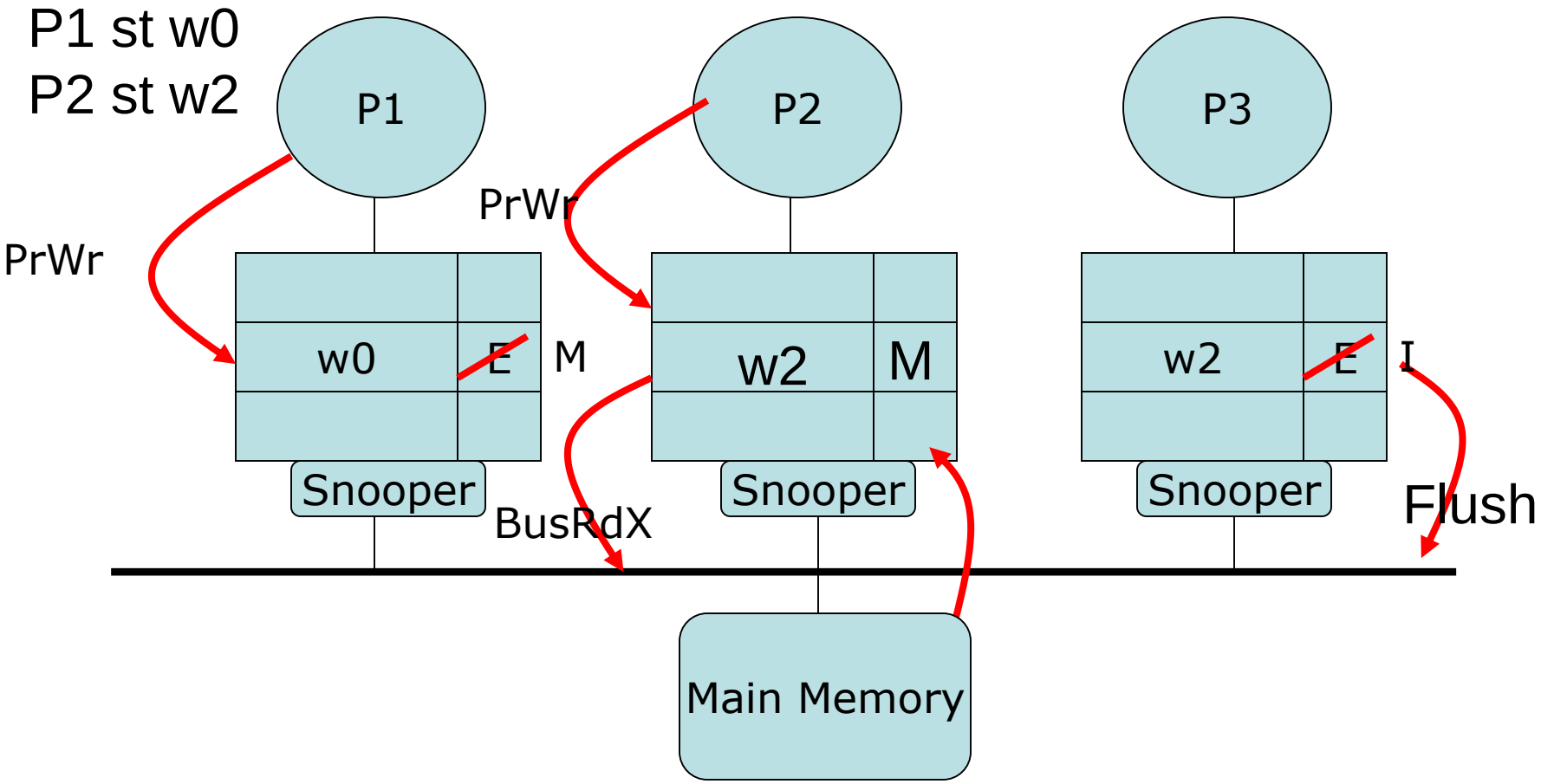
P1: ld w2

P2: ld w1

P3: ld w1

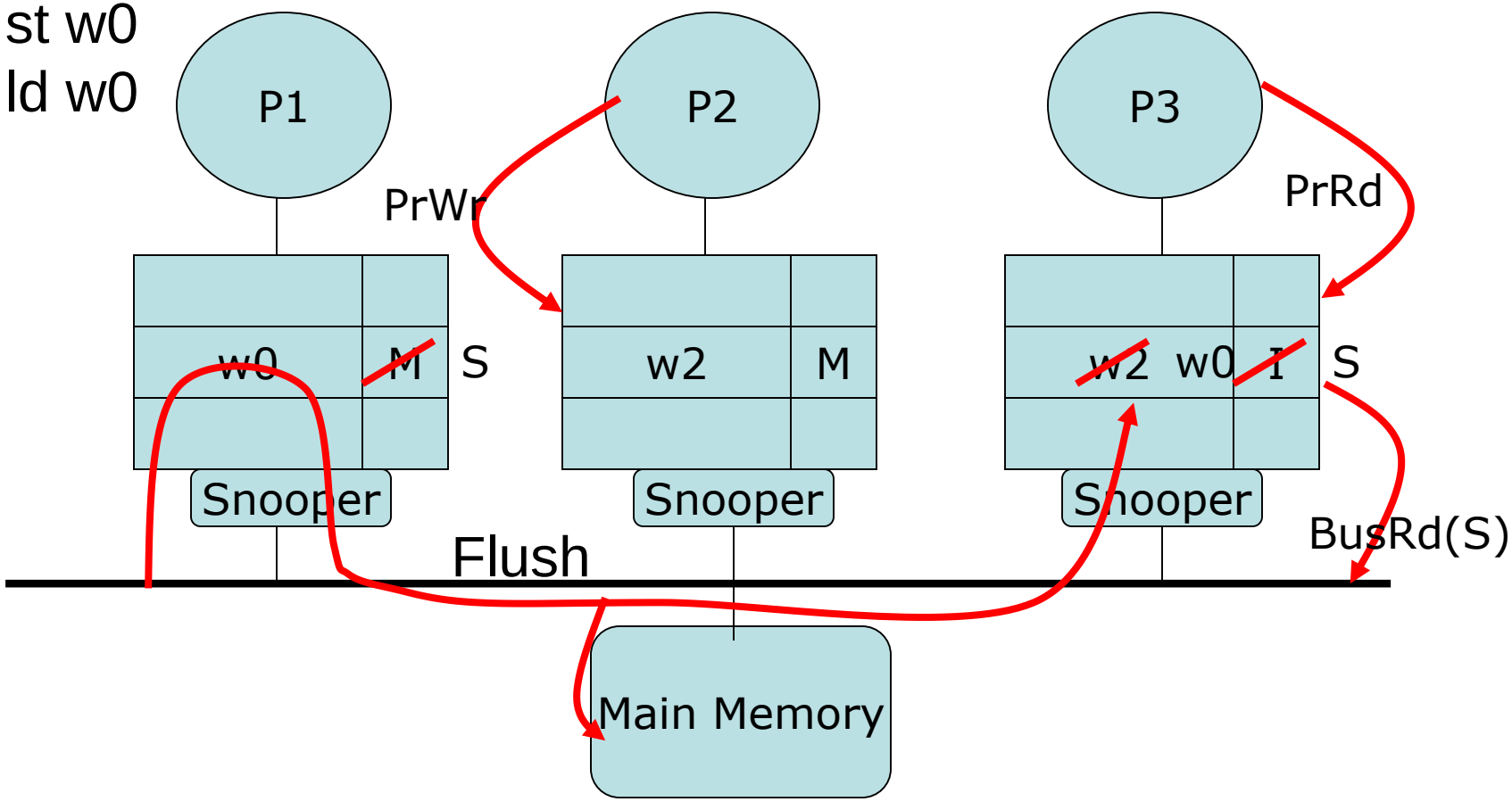


In both loads, a cache miss happens so each cache puts a BusRd onto the bus for the information. Main memory is the owner and will provide the up-to-date data. P1's cache loads w0 in the E state. P3's cache loads w2 in the E state as well.



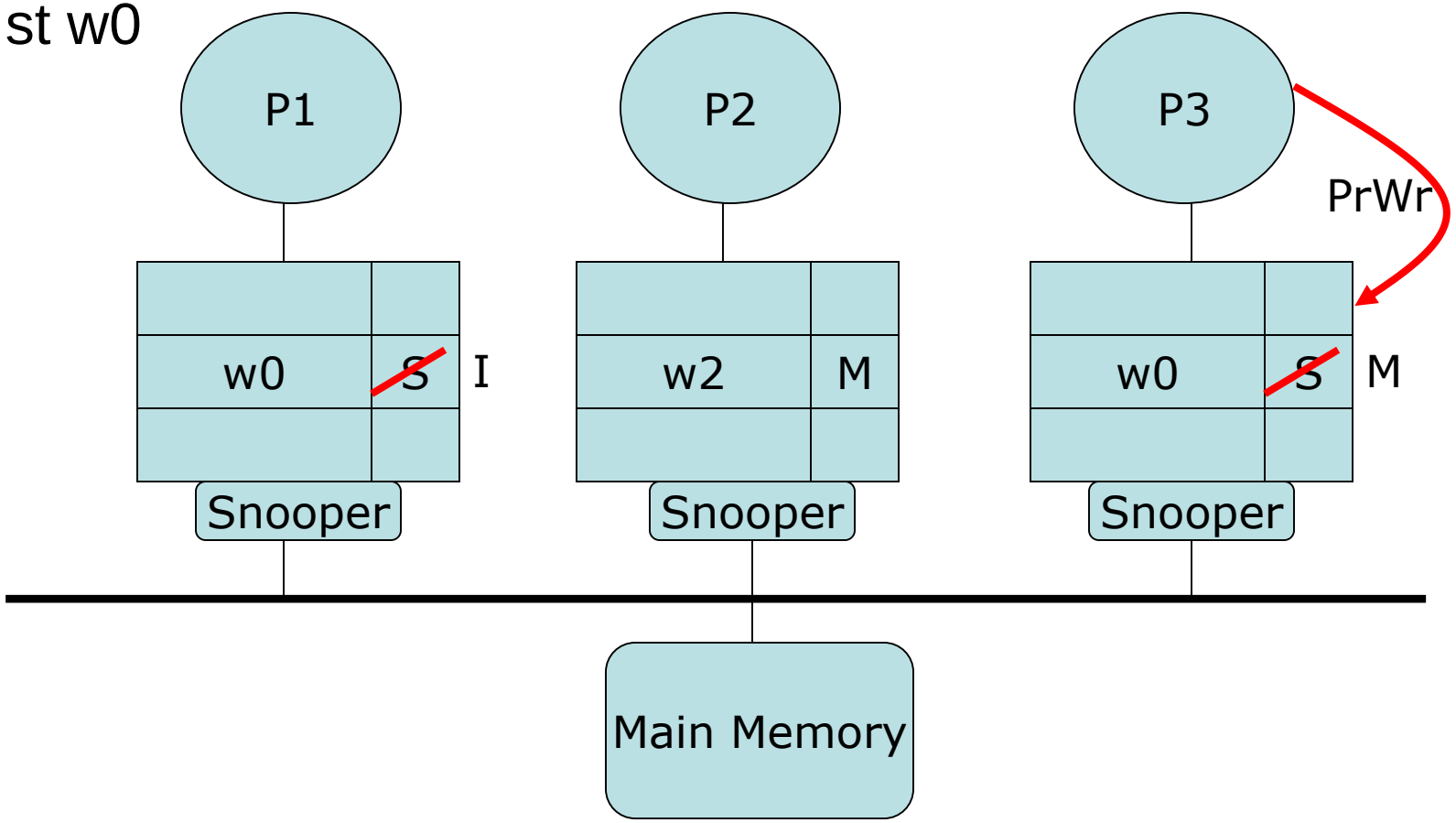
P1 has w0 in the exclusive state, so on the cache hit, it does not need to have a bus transaction. w2 is not in P2's cache, so the cache places a BusRdX to gain exclusive access. Main memory provides the data because it is not stale even though P3's cache has the data. w2 is loaded in M state and P3's cache invalidates its copy of w2.

P2 st w0
P3 ld w0

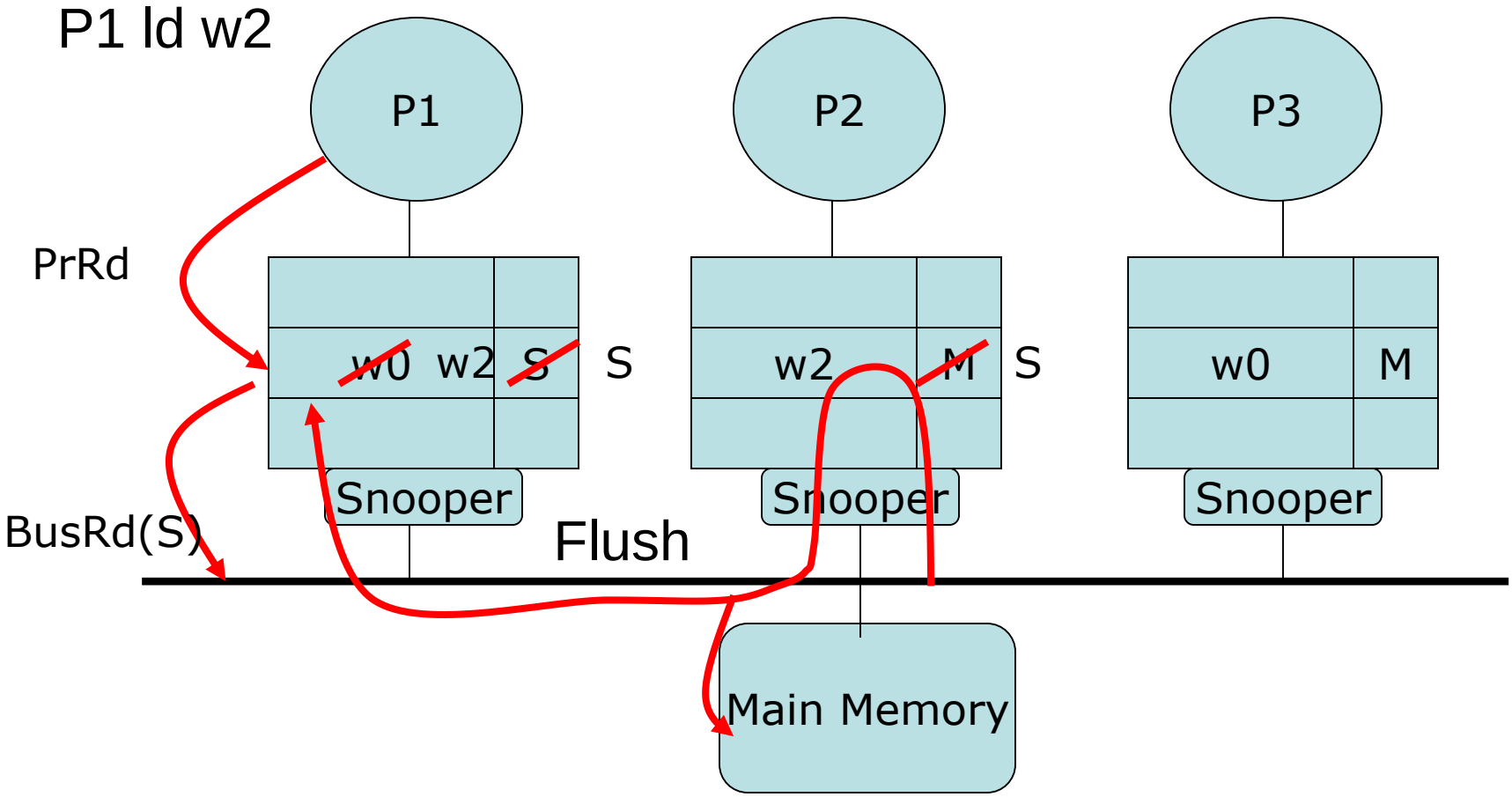


P2 executes another store to w2. It already has exclusive access to w2 and the store results in a cache hit. No bus transaction is issued by P2's cache. P3 wants to load w0. This results in a cache miss and the cache issues a BusRd transaction. P1's cache asserts the S signal because it has a dirty w0 and provides the up-to-date data through a flush. P1 changes its state to S. P3's cache loads w0 in the S state.

P3 st w0

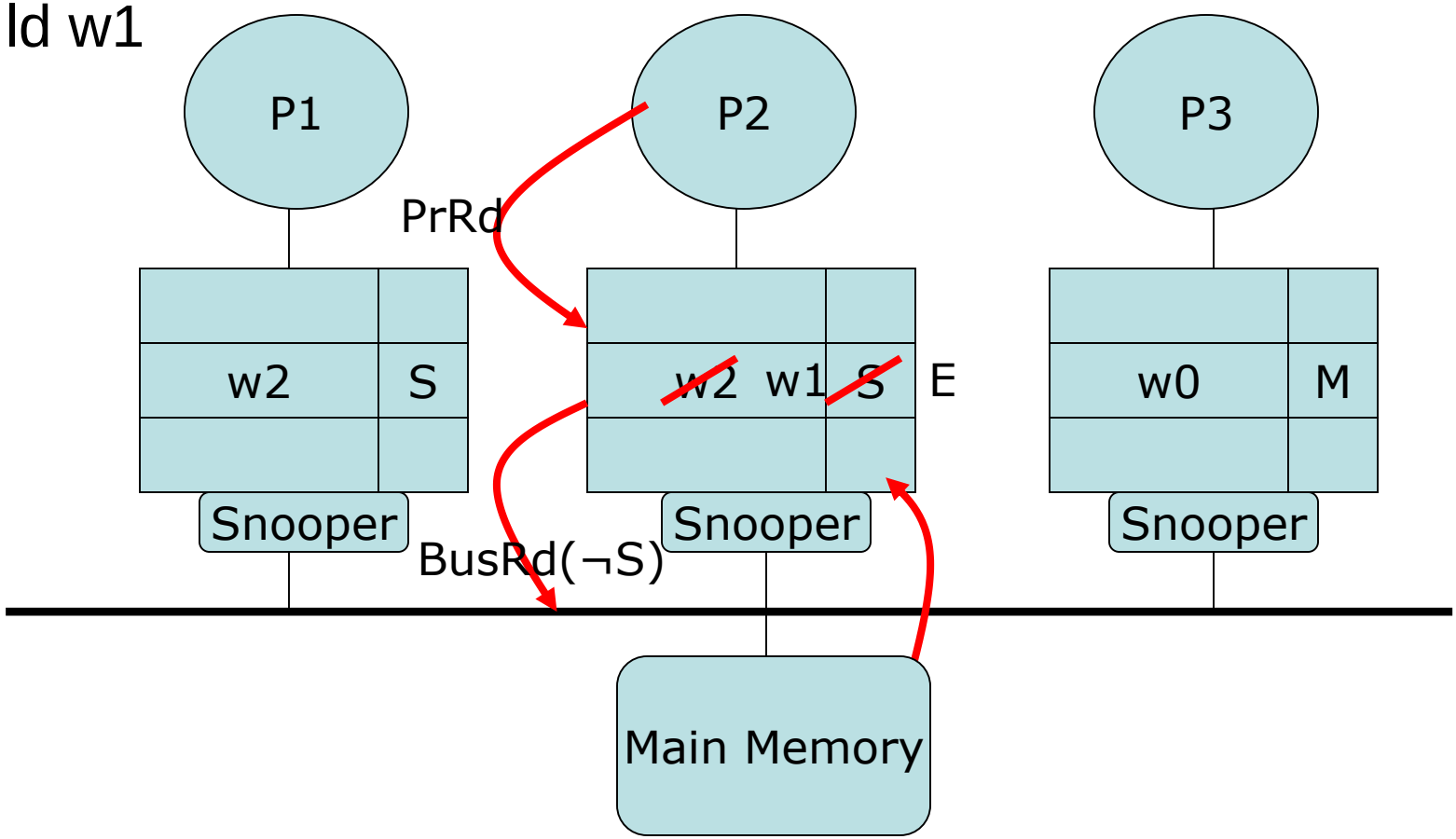


P3 executes a store to w0. Both P1 and P3 have an up-to-date, unmodified w0. What bus transactions are needed?



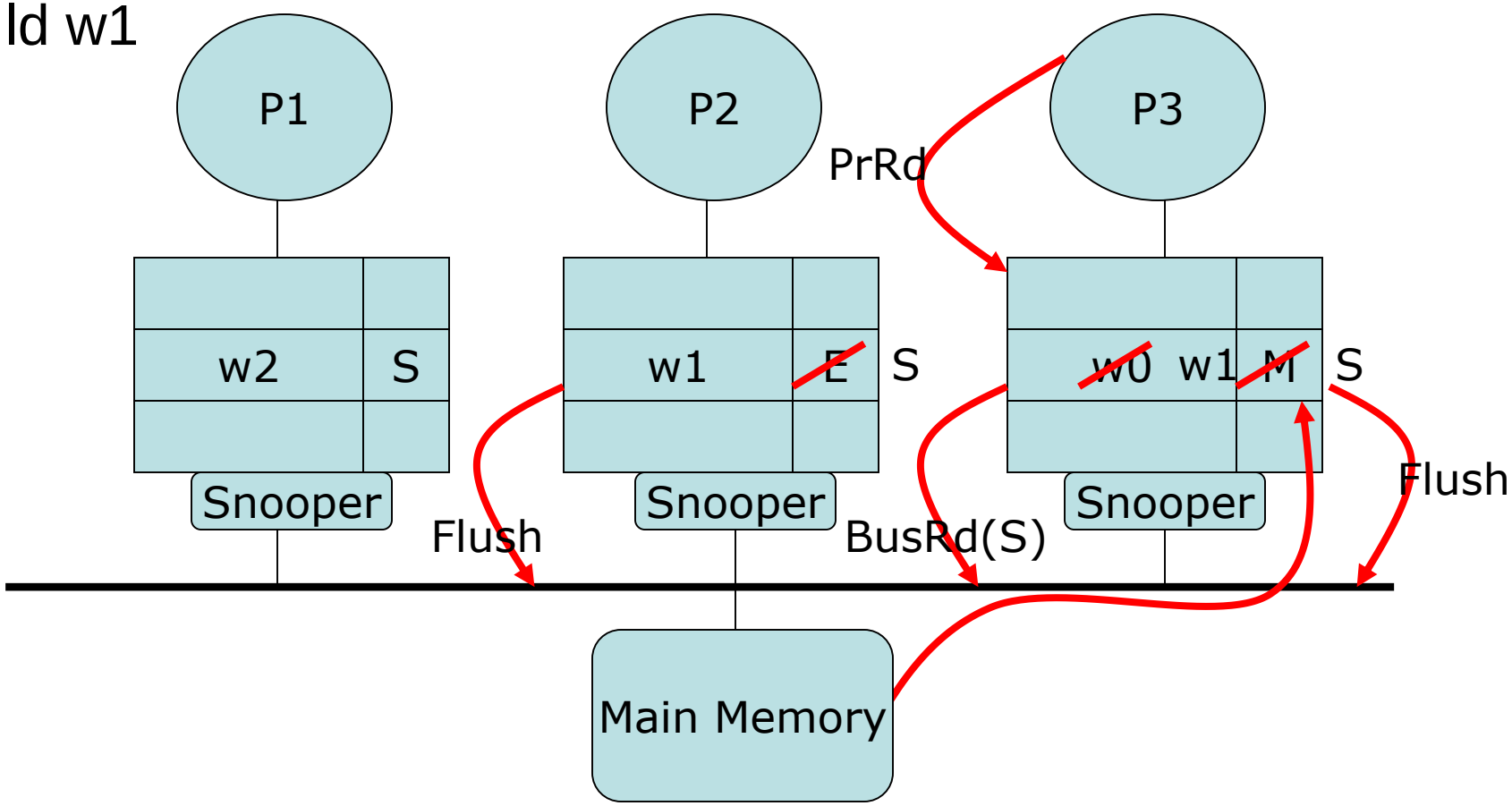
P1 wants to load w2. P1's cache does not have w2, so it issues a BusRd transaction. P2's cache turns on the S signal, so P1's cache knows to load w2 in the S state. P2's cache provides w2 for P1 and cancels the access to main memory through a Flush.

P2 ld w1



P2 wants to load w1. This generates a cache miss. P2's cache issues a BusRd transaction. The S signal is not asserted, so it knows that it has exclusive access to w1. Main memory provides the data for w1. Should the state of w2 be changed in P1 because it is the only cache that has a copy of w2?

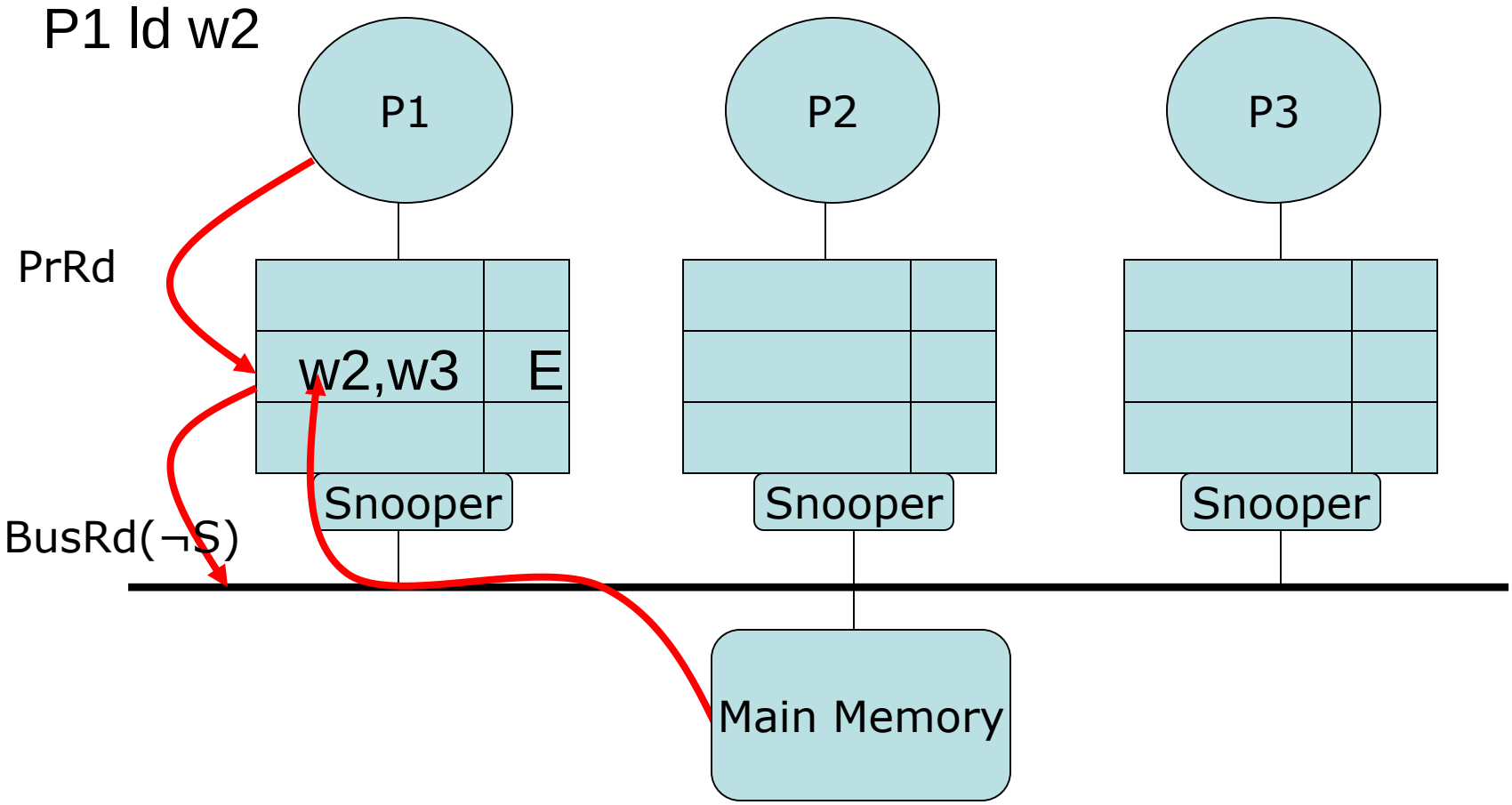
P3 ld w1



P3 wants to load w1. This generates a cache miss. P3's cache issues a BusRd transaction. The S signal is asserted by P2's cache, so P3's cache knows that it will load w1 in the S state. Main memory provides the data for w1 because its copy is not stale. P3 flushes w0 before loading w1.

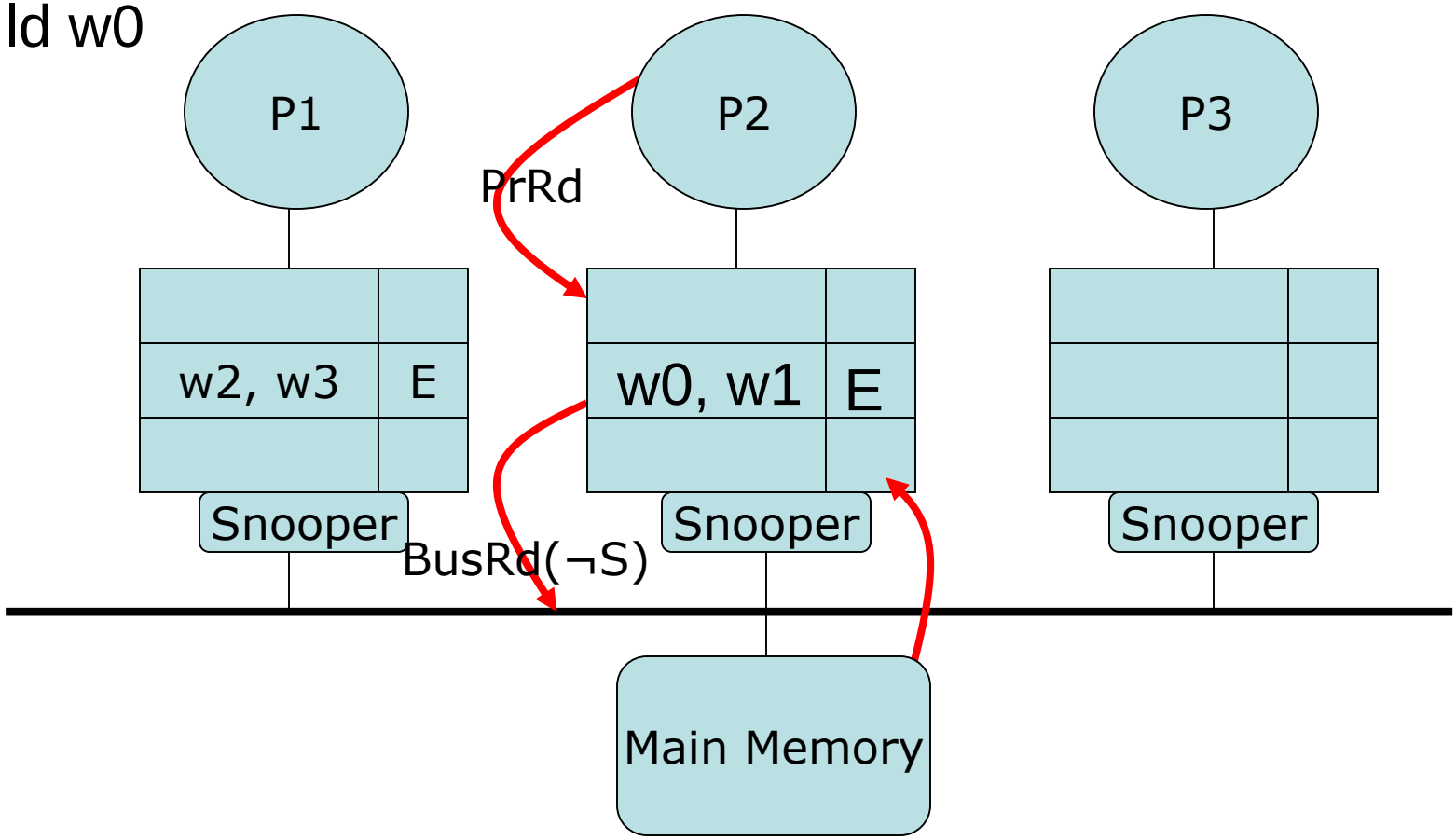
Dragon Protocol

In this system there are 3 processors. Each is loading or storing from memory locations w0, w1, w2, and w3. w0 and w1 are on the same cache line and are loaded at the same time. Likewise for w2 and w3. The two cache lines map to the same location in cache.



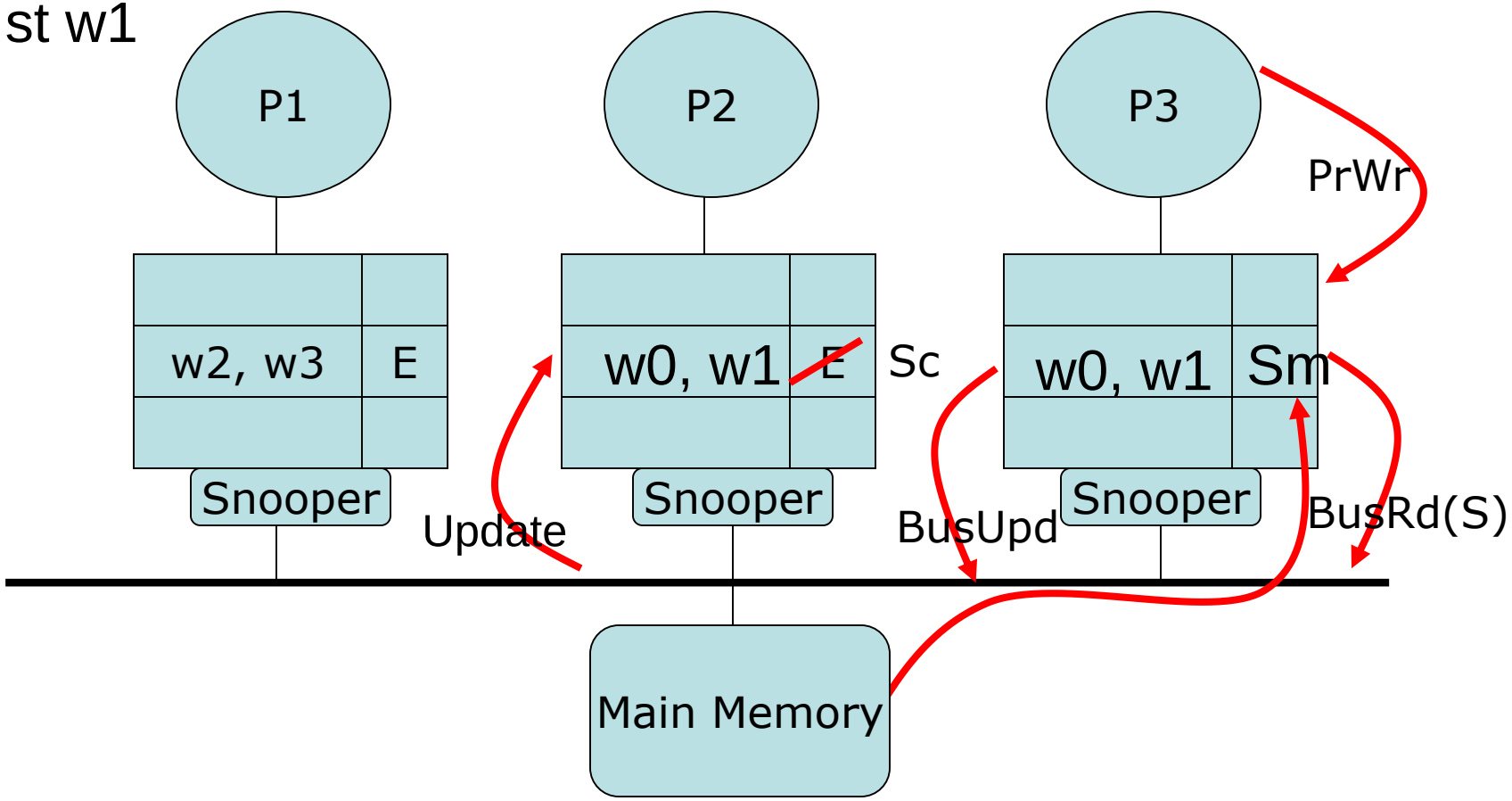
P1 wants to load w2. This generates a cache miss and P1's cache issues a BusRd bus transaction. The S signal is not asserted, so the cache knows to load w2 and w3 in the E state.

P2 ld w0

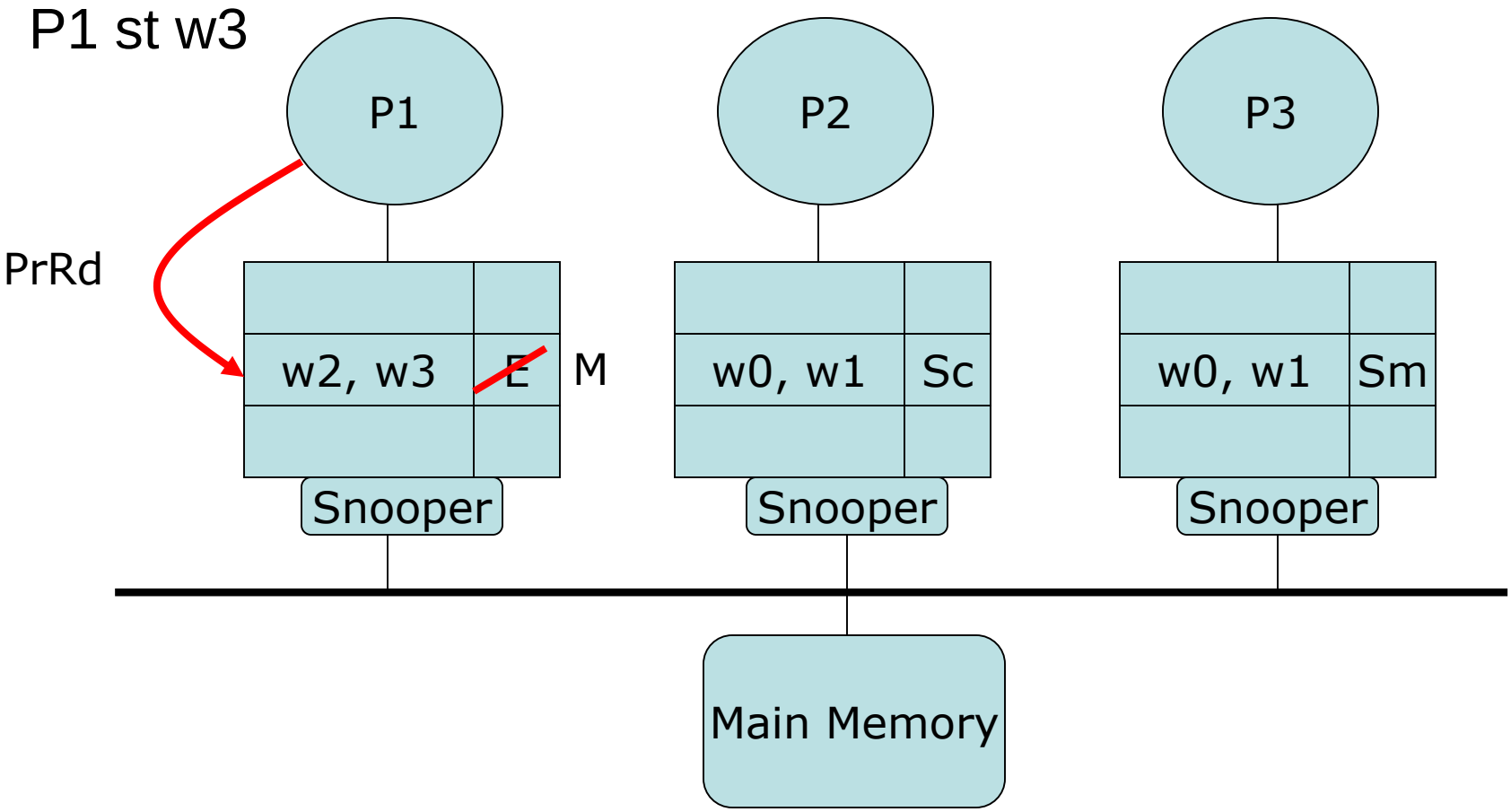


P2 wants to load w0. This generates a cache miss and P1's cache issues a BusRd bus transaction. The S signal is not asserted, so the cache knows to load w0 and w1 in the E state.

P3 st w1

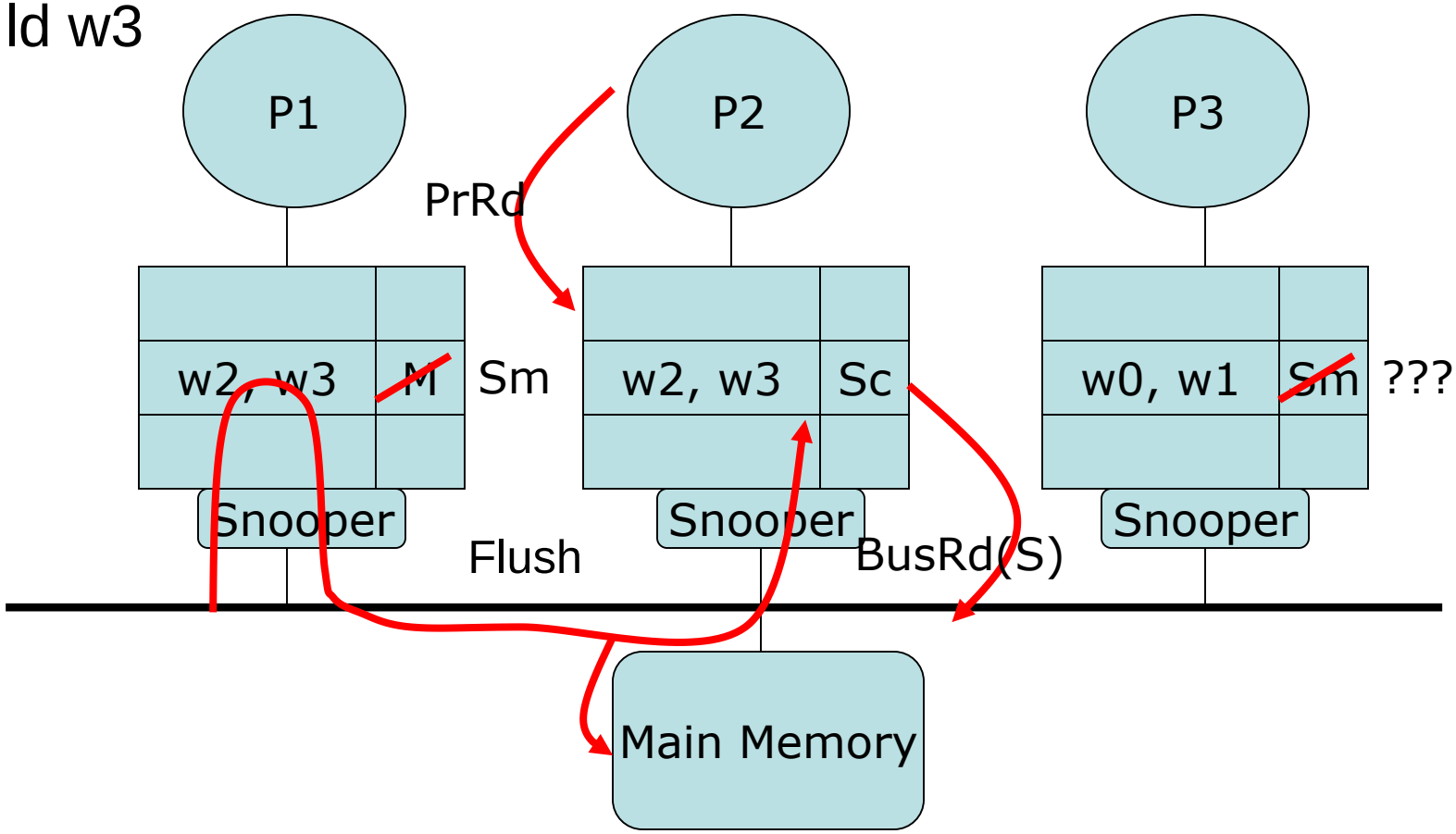


P3 wants to store w1. This generates a cache miss. Memory will provide the data as no other cache has this line in a modified state. After storing the new value of w1, P3's cache issues a BusUpd. P2 snoops this and updates its cache with the updated w1.



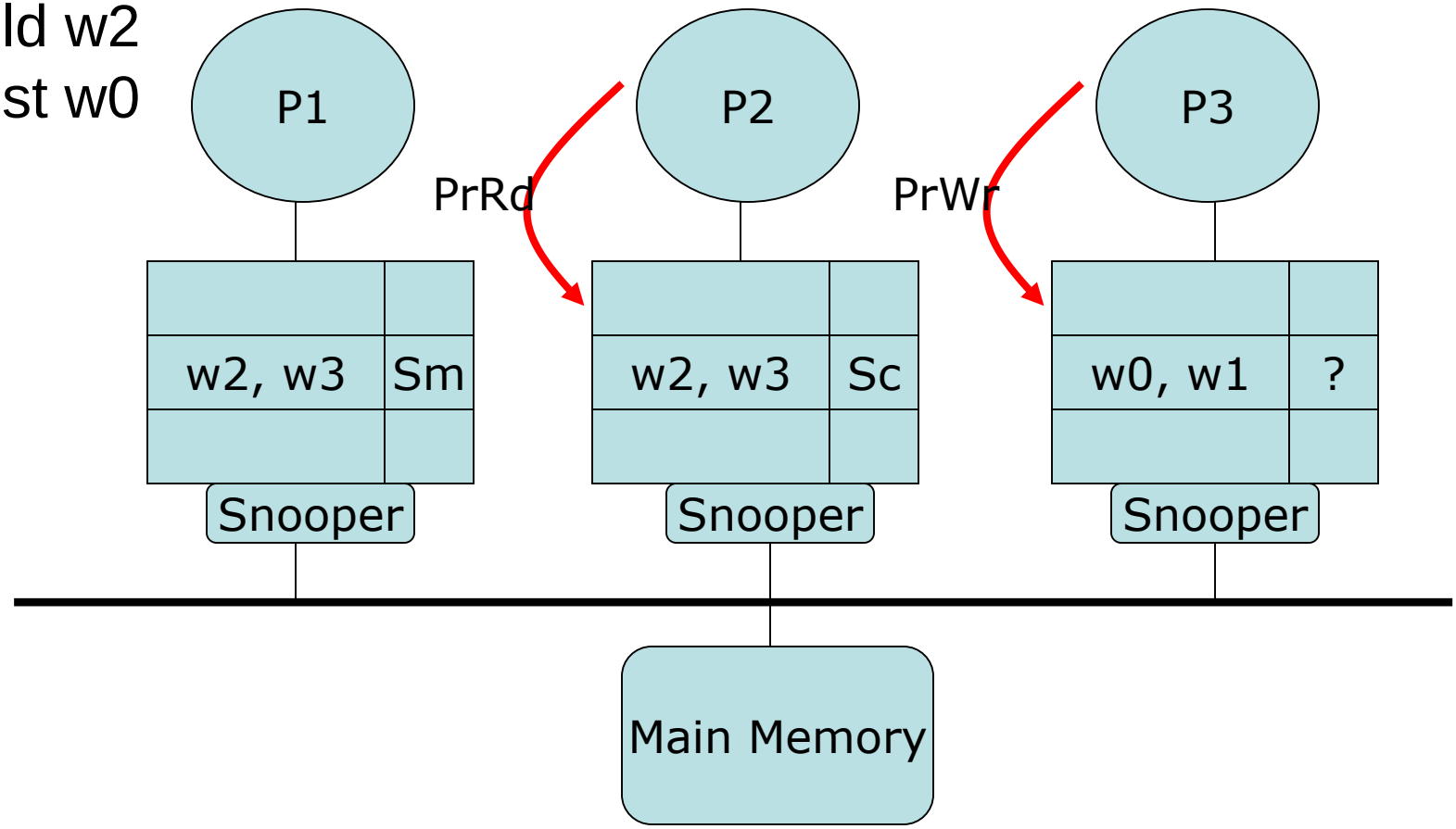
P1 issues store w3. It has exclusive access to this cache line. What bus transactions does P1's cache issue?

P2 ld w3



P2 wants to load w3. This generates a cache miss. P2's cache issues a BusRd transaction. P1 asserts the S signal, so P2 will load the cache line in Sc state. P1's cache has a modified version of the cache line, so it will provide the data for P2 with a flush transaction. P1's cache will update the line's state to Sm. Should P3 change w0/w1's state to M?

P2 ld w2
P3 st w0



P2 wants to load w2. P3 wants to store w0. What are the necessary bus transactions and cache updates that need to take place?